

**A Project Report
on**

**LIVESTOCK FARM PROTECTION FROM HARMFUL WILD ANIMALS
USING YOLOV11 WITH TEMPORAL CONSISTENCY**

Submitted to
**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY ANANTAPUR,
ANANTHAPURAMU**

In Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

Submitted By

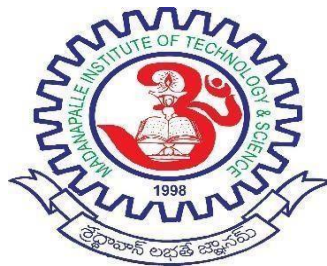
Arif Patan	—	22691A3110
Jai Surya A	—	22691A3157
Chakradhar Reddy Singam	—	22691A3124
Devan Beeda	—	22691A3130

Under the Guidance of

Dr. R. Rampriya, M.E., (Ph.D.)

Assistant Professor

Department of Computer Science and Engineering (Artificial Intelligence)



MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

(UGC – AUTONOMOUS)

(Affiliated to JNTUA, Ananthapuramu)

Accredited by NBA, Approved by AICTE, New Delhi)

AN ISO 21001: 2018 EMOS Certified Institution

P. B. No: 14, Angallu, Madanapalle – 517325

2022-2026 Batch (AY: 2025-2026)



MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

(UGC – AUTONOMOUS)

Approved by AICTE, New Delhi and Affiliated to JNTUA, Ananthapuramu

www.mits.ac.in www.mits.edu

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

BONAFIDE CERTIFICATE

This is to certify that the project work entitled “LIVESTOCK FARM PROTECTION FROM HARMFUL WILD ANIMALS USING YOLOV11 WITH TEMPORAL CONSISTENCY” is a Bonafide work carried out by

Arif Patan	–	22691A3110
Jai Surya A	–	22691A3157
Chakradhar Reddy Singam	–	22691A3124
Devan Beeda	–	22691A3130

Submitted in partial fulfillment of the requirements for the award of degree **Bachelor of Technology** in the stream of **Computer Science and Engineering (Artificial Intelligence)** in **Madanapalle Institute of Technology & Science, Madanapalle**, affiliated to **Jawaharlal Nehru Technological University Anantapur, Ananthapuramu** during the academic year 2025-2026.

Guide
Dr. R. Rampriya, M.E., (Ph.D.)
Assistant Professor
Department of CSE (AI)

Head of the Department
Dr. R. Kalpana, M.E., (Ph.D.)
Professor
Department of CSE (AI)

Submitted for the University examination held on:

Internal Examiner
Date:

External Examiner
Date:

ACKNOWLEDGEMENTS

We sincerely thank the **MANAGEMENT of Madanapalle Institute of Technology & Science** for providing excellent infrastructure and lab facilities that helped me to complete this project.

We sincerely thank **Prof. Dr. P. Ramanathan, Principal** for guiding and providing facilities for the successful completion of our project at **Madanapalle Institute of Technology & Science, Madanapalle**.

We express our deep sense of gratitude to **Dr. R. Kalpana, Professor & Head, Department of CSE(AI)** for her continuous support in making necessary arrangements for the successful completion of the project.

We express our deep gratitude to my guide, **Dr. R. Rampriya, Asst. Professor, Department of CSE(AI)** for her guidance and encouragement that helped us to complete this project.

We express our deep gratitude to **Dr. Vamsi Bandi and Dr. G. Jenifa, Project Coordinator(s), Department of CSE(AI)** for their valuable guidance and encouragement that helped us to complete this project.

We also wish to place on record my gratefulness to other **Faculty members of Department of CSE(AI)** and also to our friends and our parents for their help and cooperation during our project work.



MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

(UGC-AUTONOMOUS INSTITUTION)

Affiliated to JNTUA, Ananthapuramu & Approved by AICTE, New Delhi
NAAC Accredited with A+ Grade, NIRF India Rankings 2024-Band: 201-300 (Engg.)
NBA Accredited -B.Tech. (CIVIL, CSE, ECE, EEE, MECH, CST), MBA & MCA
www.mits.ac.in



RECOGNISED RESEARCH CENTER

Plagiarism Verification *Certificate*

This is to certify that the B. Tech Project report titled, “**Livestock Farm Protection from Harmful Wild Animals Using Yolov11 with Temporal Consistency**” submitted by **ARIF PATAN (22691A3110), JAI SURYA A (22691A3157), SINGAM CHAKRADHAR REDDY (22691A3124), DEVAN BEEDA (22691A3130)** has been evaluated using **Anti- Plagiarism Software, TURNITIN** and based on the analysis report generated by the software, the report’s similarity index is found to be ---%.

The following is the TURNITIN report for the project report consisting of --- pages.

DECLARATION

We hereby declare that the results embodied in this project “**Livestock Farm Protection from Harmful Wild Animals Using Yolov11 with Temporal Consistency**” by us under the guidance of **Dr. R. Rampriya, Asst. Professor, Department of Computer Science and Engineering (Artificial Intelligence)** in partial fulfillment of the award of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)** from **Jawaharlal Nehru Technological University Anantapur, Ananthapuramu**. The content of this report is not submitted to any other University/Institute for award of any other degree.

Date :

Place : Madanapalle

PROJECT ASSOCIATES:

ARIF PATAN

JAI SURYA A

CHAKRADHAR REDDY SINGAM

DEVAN BEEDA

I certify that the above statement made by the students is correct to the best of my knowledge.

Date:

Guide

Dr. R. Rampriya

INDEX

S.NO	TOPIC	PAGE NO.
1.	INTRODUCTION	
	1.1 Introduction to Harmful Wild Animal Detection	2
	1.2 Motivation	2
	1.3 Problem Definition	3
	1.4 Impact of Wildlife Intrusion on Agriculture and Residential Areas	4
	1.5 Role of Deep Learning and Computer Vision in Wildlife Monitoring	4
	1.6 Objective of the Project	5
	1.7 Human–Wildlife Conflict Statistics	6
	1.8 Limitations of the Project	6
	1.9 Organization of Documentation	7
2.	LITERATURE SURVEY	
	2.1 Introduction	9
	2.2 Existing Wildlife Monitoring Systems	9
	2.3 Disadvantages of Existing System	19
	2.4 Proposed System	19
	2.5 Advantages of the Proposed System	22
3.	ANALYSIS	
	3.1 Introduction	24
	3.2 Software Requirement Specification	24

4.	DESIGN	
	4.1 Introduction	30
	4.2 UML Diagrams	31
	4.3 Module Design and Organization	38
	4.4 Summary	38
5.	IMPLEMENTATION AND RESULTS	
	5.1 Introduction	40
	5.2 Preparation and Preprocessing of the Dataset	41
	5.3 Method of Implementation	44
	5.4 Output Screens and Result Analysis	107
	5.5 Summary	112
6.	TESTING AND VALIDATION	
	6.1 Introduction	115
	6.2 Test Case Design for Video	117
	6.3 Model Validation on Test Dataset	121
	6.4 Performance Metrics Analysis	122
	6.5 Prediction Outcomes	123
	6.6 Summary	127
7.	CONCLUSION	
	7.1 Conclusion	130
	7.2 Future Enhancements	130
	REFERENCES	131

ABSTRACT

Wildlife intrusion into livestock farms located in forest-adjacent and rural regions poses serious economic, ecological, and safety challenges, necessitating reliable automated surveillance solutions. This paper presents a spatiotemporal wild animal detection framework based on the YOLOv11 deep convolutional object detection architecture, augmented with a temporal consistency enforcement module to enhance detection stability in continuous video streams. The proposed system performs high-speed frame-level inference to localize and classify multiple wild animal species while addressing common video surveillance challenges such as motion-induced blur, scale variation, partial occlusion, illumination fluctuations, and background clutter. To mitigate frame-to-frame prediction inconsistency and spurious detections, a temporal smoothing strategy is incorporated that exploits inter-frame confidence propagation and temporal aggregation of bounding box predictions. This mechanism effectively reduces false positives and detection flickering by enforcing temporal coherence across consecutive frames without modifying the core network configuration. The conceptual framework is designed to carry out live inference while remaining computationally lightweight, making it suitable for deployment on resource-constrained edge hardware without compromising detection performance. Experimental results demonstrate notable improvements in detection robustness, minority-class recall, and false alarm reduction compared to a baseline YOLOv11 model, highlighting the effectiveness of integrating temporal consistency into real-time wildlife surveillance systems.

Keywords: *YOLOv11, Spatiotemporal Object Detection, Wildlife Intrusion Monitoring, Temporal Consistency Modeling, Video Surveillance, Deep Neural Networks*

LIST OF FIGURES

Figures	Title	Page
2.1	Proposed Architecture for Harmful Wild Animal Detection using YOLOV11	20
4.1	State Chart Diagram	33
4.2	Sequence Diagram	35
4.3	Use Case Diagram	36
4.4	Collaborative Diagram	37
5.1	Confusion Matrix for Wild Animal Detection Model	96
5.2	User Interface of the Livestock Farm Protection System for Video Upload and Animal Detection	108
5.3	Video Upload Interface with Selected Surveillance Video for Wildlife Detection	109
5.4	Detection Result Showing Identified Wild Animal with Bounding Box and Email Alert Notification	109
5.5	YOLO vs YOLO + Temporal Consistency Interface for Video Upload	110
5.6	Video Selection Interface for YOLO and Temporal Consistency Detection Comparison	111
5.7	Detection Comparison Result Showing Base YOLO Detection and Temporal Consistency Verification	112
6.1	YOLOv11 Training and Validation Performance Analysis	123
6.2	Animal Detection Results	126
6.3	Wildlife Ground Truth Labels	126
6.4	Wildlife Ground Truth Labels	127

LIST OF TABLES

Tables	Title	Page
2.1	Summary of Existing Research Works	13
6.1	Test Case for model training	117
6.2	Test Case for Dataset Annotation	118
6.3	Test Case for preprocessing and model training	118
6.4	Test Case for model Generation	119
6.5	Test Case for Dataset Balancing	119
6.6	Test Case for Video Detection Stability	120
6.7	Test Case for Temporal Consistency	120
6.8	Test Case for System Interface	121
6.9	Prediction outcomes	122
6.10	Strengths and Weaknesses of System Components	125

CHAPTER-1

INTRODUCTION

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION TO HARMFUL WILD ANIMAL DETECTION

Human settlements located near forest regions and agricultural lands often face challenges due to the unexpected movement of wild animals into inhabited areas. Such intrusions can cause significant damage to crops, livestock, property, and in some cases, pose serious risks to human life. The increasing overlap between wildlife habitats and human environments has intensified the need for reliable monitoring and early warning systems.

Traditional methods for preventing animal intrusion, such as manual surveillance, fencing, and alarm systems, are often inefficient, labor-intensive, or expensive to maintain. These approaches also lack real-time responsiveness and may fail under poor lighting or adverse weather conditions. Consequently, the need for smart and automated systems has been increasing, especially those capable of operating continuously while requiring very little human involvement.

Recent progress in computer vision and deep learning has created new opportunities for building automated wildlife monitoring systems. Object detection algorithms, especially those developed using the YOLO (You Only Look Once) architecture, allow animals to be detected and recognized quickly and accurately in both images and video streams. By analyzing live camera feeds, these systems can detect the presence of harmful animals and trigger alerts instantly.

Harmful wild animal detection systems aim not only to identify animals but also to ensure stable and reliable predictions in real-time environments. The techniques such as temporal consistency reduce the false alarms by ensuring the detections are consistent in consecutive frames. The improvement in the reliability of the system is beneficial in real-world scenarios in agricultural fields, rural areas, and even near forest regions.

In general, intelligent wildlife detection systems help reduce economic damage, enhance public safety, and promote a more balanced coexistence between humans and wildlife.

1.2MOTIVATION

This project is motivated by the growing number of cases where wild animals enter agricultural lands, livestock farms, and rural residential areas, causing damage and safety concerns. Farmers and people living close to forest areas often face crop loss, attacks on livestock, and damage to property when animals like wild boars, elephants, monkeys, and other species enter nearby lands

unexpectedly. These incidents not only result in economic losses but also create fear and safety concerns among local communities.

Conventional prevention methods, including fencing, manual guarding, and basic alarm systems, often fail to provide reliable protection. They require continuous human supervision, are costly to maintain, and may not function effectively during nighttime or adverse weather conditions. In many cases, delayed detection leads to irreversible damage before any preventive action can be taken.

Recent progress in artificial intelligence and computer vision has opened new possibilities for building automated wildlife monitoring systems capable of identifying animals in real time. Among the available detection architectures, YOLO-based models have gained wide adoption due to their balance of speed and accuracy. Despite this, processing individual frames in isolation often leads to inconsistent or erroneous predictions when applied to continuous video footage.

This project focuses on developing a robust surveillance framework that combines state-of-the-art object detection with temporal stabilization techniques to achieve dependable wildlife identification across consecutive video frames. The aim is to reduce false alarms, improve the reliability of the detection process, and provide timely alerts, which will be helpful for protecting farms and rural communities.

1.3 PROBLEM DEFINITION

Wild animal intrusion into agricultural lands, livestock farms, and rural residential areas has become a serious concern in regions located near forests and wildlife habitats. Animals entering these areas often cause crop damage, livestock loss, infrastructure destruction, and potential threats to human safety. The absence of timely detection and response mechanisms increases economic losses and heightens the risk of human–wildlife conflict.

Existing monitoring approaches such as manual surveillance, fencing systems, and traditional alarm devices are either labour-intensive, unreliable, or ineffective in low-light and dynamic environmental conditions. Moreover, many vision-based systems rely solely on frame-by-frame detection, which may result in unstable predictions, false positives, or missed detections in real-time video streams.

Therefore, the core problem is to develop an automated, accurate, and real-time detection system capable of identifying harmful wild animals in continuous video feeds while minimizing false alarms. The system should operate efficiently under varying environmental conditions and provide reliable alerts when an intrusion is confirmed. Addressing this problem requires the integration of

advanced object detection models with mechanisms that ensure temporal stability and consistent performance in practical deployment scenarios.

1.4 IMPACT OF WILDLIFE INTRUSION ON AGRICULTURE AND RESIDENTIAL AREAS

Wildlife intrusion into agricultural and residential regions has significant economic, social, and environmental consequences. In agricultural areas, animals such as wild boars, elephants, deer, and monkeys frequently damage crops at various stages of growth. This results in reduced yield, financial loss for farmers, and instability in local food supply chains. In severe cases, repeated intrusions discourage farmers from cultivating certain crops, affecting long-term agricultural productivity.

Livestock farms are also vulnerable to attacks by predatory animals. The loss of cattle, goats, poultry, or other farm animals directly impacts farmers income and livelihood. Beyond economic damage, such incidents create psychological stress and fear among rural communities, especially during nighttime when monitoring is limited.

In residential areas located near forest boundaries, wildlife intrusion can lead to property damage and pose safety risks to humans. Encounters between humans and wild animals may result in injuries or fatalities on both sides. Increasing human expansion into natural habitats has intensified these conflicts, making preventive monitoring systems more necessary than ever.

Therefore, minimizing wildlife intrusion is not only important for protecting agricultural investments and residential safety but also for promoting a balanced coexistence between humans and wildlife. Automated detection frameworks can significantly contribute to minimizing such adverse effects by enabling prompt alerts and facilitating timely preventive action.

1.5 ROLE OF DEEP LEARNING AND COMPUTER VISION IN WILDLIFE MONITORING

Rapid developments in deep learning and computer vision over recent years have substantially enhanced the capability to build and deploy effective wildlife monitoring solutions. Conventional systems for monitoring wildlife often rely on human observation, camera trap results, or sensor triggers, and these processes are often labour-intensive and subject to errors. Computer vision lets machines process images and videos automatically, so they can quickly recognize and sort out different animals.

Among modern machine learning approaches, convolutional neural networks have

demonstrated a strong ability to autonomously extract and recognize intricate visual patterns and spatial features directly from raw image data. Modern object detection algorithms such as the YOLO series of algorithms are quite efficient in detecting objects in areas such as farmland and near forest regions. These models can identify multiple animal species within a single frame while maintaining real-time processing capability.

Furthermore, deep learning techniques improve adaptability to varying environmental conditions such as low lighting, motion blur, occlusion, and background clutter. When used with video streams, incorporating temporal data improves the consistency of detections and helps minimize incorrect predictions.

In general, combining computer vision with deep learning techniques offers a smart, scalable, and efficient approach for continuous wildlife surveillance, timely detection of animal intrusions, and automatic alert notifications.

1.6 OBJECTIVES OF THE PROJECT

The objectives of this project are organized into five stages, which are described below:

1. To design and develop a real-time video monitoring system that protects livestock farms and nearby residential areas by detecting harmful wild animals using the YOLOv11 object detection model.
2. To enhance detection stability in continuous video streams by integrating a temporal consistency mechanism that reduces frame-to-frame fluctuations and false alarms.
3. To evaluate how well the proposed method works, we compare frame-based detection with temporally consistent detection by looking at accuracy, stability, and how effectively each reduces false alarms.
4. The proposed method is evaluated by comparing single-frame detection with temporally consistent detection, with emphasis on accuracy, stability, and minimizing false alarms.
5. To implement an interactive user interface that allows users to upload surveillance videos and visualize detected animals along with temporally stabilized predictions for effective monitoring and decision-making.

1.7 HUMAN-WILDLIFE CONFLICT STATISTICS

Human–wildlife conflict has become a growing concern worldwide due to habitat loss, urban expansion, and agricultural encroachment into forest regions. According to global conservation reports, thousands of conflict incidents are recorded annually, leading to significant crop losses, livestock deaths, and injuries to both humans and animals. In many developing countries, wildlife intrusion is one of the major causes of agricultural damage, affecting rural livelihoods and food security.

In India, human–wildlife conflict is particularly prominent due to the close proximity of villages and farmlands to forest areas. Government data and wildlife conservation agencies report frequent cases of crop destruction by wild boars and elephants, livestock attacks by leopards and tigers, and occasional human casualties. States with extensive forest cover, such as Karnataka, Kerala, Tamil Nadu, Assam, and West Bengal, experience a higher number of such incidents. Compensation claims filed by farmers each year reflect the economic burden caused by wildlife intrusion.

The increasing trend of these incidents highlights the urgent need for automated monitoring and early warning systems. Implementing intelligent detection technologies can help reduce losses, improve rural safety, and support better coexistence between humans and wildlife.

1.8 LIMITATIONS OF THE PROJECT

Even though the wildlife detection system works well in regular situations, its accuracy is heavily influenced by how good the input video is. Poor-quality or low-resolution footage can significantly reduce detection performance, as important visual features may not be clearly captured. Environmental factors such as heavy rain, fog, poor illumination, or nighttime conditions can further affect visibility and lead to missed detections or lower confidence predictions. Additionally, animals appearing at very small sizes within the frame, especially when captured from a distance, may not be detected accurately due to limited pixel information.

Another limitation is that the current model is trained to recognize only 13 predefined harmful wild animal classes. However, since there are many other species that may pose threats, the system is unable to recognize any animal outside the trained species without the collection of more data. Therefore, while the system is useful and efficient in monitoring objects in real time, it is possible to improve the effectiveness of the system by using more data, improving the quality of the video, and making it work in many environments.

1.9 ORGANIZATION OF DOCUMENTATION

The complete project documentation is systematically organized into seven chapters covering the introduction, literature survey, analysis, design, implementation, testing, and conclusion of the proposed harmful wild animal detection system.

Chapter 1: Introduction presents the background of harmful wild animal detection, motivation behind the work, problem definition, impact of wildlife intrusion on agriculture and residential areas, role of deep learning and computer vision, objectives of the project, human–wildlife conflict statistics (worldwide and India), limitations of the project, and overall organization of the documentation.

Chapter 2: A literature review gives a detailed look at past research on tracking wildlife and systems used to detect animals entering restricted areas. It discusses existing methodologies, their disadvantages, and introduces the proposed system based on YOLOv11 integrated with temporal consistency, along with its advantages.

Chapter 3: Analysis explains the system requirements and project specifications. It includes the Software Requirement Specification (SRS) covering functional and non-functional requirements, hardware and software requirements, and necessary tools and packages.

Chapter 4: Design describes the system architecture, module design, and overall organization of the proposed model. Architectural diagrams and structural design details are presented to explain the workflow of the system.

Chapter 5: This covers the end-to-end implementation of the proposed system, beginning with dataset preparation and preprocessing procedures, followed by the training and integration of the YOLOv11 model with the temporal consistency mechanism. It further presents the visual outputs of the detection results along with a quantitative assessment of the system's performance using evaluation metrics including precision, recall, F1-score, and mAP.

Chapter 6: The Testing and Validation section discusses how the system is tested using image and video datasets. This chapter addresses model validation conducted on unseen test data, an analysis of key performance indicators including precision, recall, mAP@50, and mAP@50-95, along with inference results on video inputs and the corresponding alert generation mechanism.

Chapter 7: Conclusion summarizes the overall work carried out in the project and discusses possible future enhancements to improve system performance and scalability.

Finally, the References section lists all research papers, journals, and sources referred to during the development of this project.

CHAPTER-2

LITERATURE SURVEY

CHAPTER-2

LITERATURE SURVEY

2.1 INTRODUCTION

Conflicts between humans and wild animals have risen significantly in recent years, mainly driven by habitat loss, rapid urbanization, and the expansion of agricultural activities. As natural ecosystems continue to decline and farmlands spread into forested regions, wildlife is compelled to venture closer to human-inhabited areas in search of food and safe shelter. This increased interaction leads to complex challenges, posing threats to both community safety and wildlife survival. Farmers suffer financial losses due to crop destruction, livestock attacks, and property damage, while animals may face injury, capture, or retaliatory harm from local communities. In many rural areas, such encounters occur frequently during nighttime when animals move toward cultivated lands or water sources. Consequently, it becomes increasingly important to develop effective monitoring and prevention strategies that can reduce these conflicts while maintaining ecological balance. Therefore, implementing preventive and non-invasive monitoring solutions is essential to ensure sustainable coexistence between humans and wildlife.

Human–wildlife conflict not only affects livelihoods but also has long-term environmental consequences. When wildlife populations are disturbed or displaced from their natural habitats, ecosystems may experience imbalance because many animals play crucial roles in maintaining biodiversity. For instance, herbivores influence vegetation patterns, while predators help regulate prey populations. If conflicts result in declining wildlife populations, these ecological processes may be disrupted. At the same time, communities living near forest boundaries often depend heavily on agriculture for income, making them particularly vulnerable to wildlife intrusions. As a result, solutions that combine technological innovation with conservation awareness are necessary to address both human safety and wildlife protection.

2.2 EXISTING WILDLIFE MONITORING SYSTEM

Delwar and colleagues designed a real-time agricultural monitoring system that integrates IoT sensors with a YOLOv8-based detection model to identify animal intrusions. Their architecture utilized edge-mounted cameras along with cloud-based alert services to ensure prompt notification and response. The findings demonstrated that coupling deep learning detection with IoT infrastructure enhances monitoring efficiency in farm environments.

Wang and co-authors proposed CF-YOLO, an enhanced variant derived from YOLOv11, specifically optimized for detecting small objects in drone-captured imagery. By incorporating tailored feature fusion modules, the model improved the representation of fine-grained features, resulting in better performance for tiny object detection compared to the standard YOLOv11 baseline.

Saoud and collaborators developed a wildlife conservation framework that integrates YOLOv10 with MobileSAM to achieve real-time detection and segmentation of Houbara birds. By combining object detection with lightweight segmentation, their system achieved more precise localization. The study highlighted the value of efficient segmentation techniques in conservation-focused monitoring applications.

Kline and team introduced an edge-oriented adaptive drone platform for wildlife observation. Their system dynamically modified detection settings according to environmental changes, showing that intelligent monitoring approaches are capable of running efficiently on edge hardware without strong dependence on centralized computing systems.

Kumari and team presented a review on deep learning methods used for detecting wild animal activities. They compared CNN-based detection frameworks and discussed dataset challenges in wildlife monitoring. The study identified real-time inference and robustness as major research gaps.

The Authors of JETIR proposed a wild animal detection and alarm system using deep learning in the context of rural areas. The alarm was generated based on the detection of intrusions in the farmland. However, the main aim of the paper was the accuracy of the detection rather than the stability of the video stream.

Jaiswal and Busi Reddy designed a system based on deep learning to identify large animals on roads, aiming to reduce the risk of vehicle–animal collisions. Their model improved detection reliability in complex traffic scenarios. The research emphasized safety applications of wildlife detection.

MASF-YOLO was proposed as a modified variant of the YOLOv11 architecture, specifically optimized to improve detection performance for small-scale targets captured through unmanned aerial vehicles. Their approach focused on refining the feature extraction stages to better capture subtle and fine-grained visual details. These improvements led to notable gains in detecting small-scale targets compared to the standard YOLOv11 configuration.

Moghadami and colleagues introduced a modified YOLO-based framework aimed at improving both detection accuracy and computational efficiency for small objects. Their method incorporated architectural refinements designed to lower processing complexity while maintaining strong predictive performance.

Jung developed YOLO-Drone using the Ghost Head network to optimize object detection in aerial imagery. The model reduced parameter complexity while maintaining competitive accuracy. The approach demonstrated suitability for lightweight drone-based surveillance.

Peng et al. presented YOLOv11-Litchi for UAV-based fruit detection. The study focused on agricultural object detection using optimized backbone modules. Their results highlighted the adaptability of YOLOv11 for precision farming applications.

IET Research explored IoT-based health surveillance systems for livestock monitoring. The framework integrated sensor networks with real-time data analytics. Although focused on livestock health, the study emphasized the importance of smart agricultural monitoring systems.

ICDICI Proceedings discussed automated camera trap systems using temporal data and depth estimation for wildlife monitoring. The approach leveraged spatiotemporal information to enhance detection reliability. The research highlighted the significance of temporal information in wildlife analysis.

Kundu and collaborators developed a real-time anomaly detection framework aimed at identifying deforestation activities using YOLO-based object detection integrated with intelligent agent systems. Their approach leveraged computer vision techniques to continuously observe environmental variations and automatically flag unusual changes. The research emphasized the increasing importance of automated vision-based approaches for large-scale ecological monitoring.

The study titled Deep Learning for Dynamic Wildlife Monitoring presented a comprehensive real-time framework that incorporated spatiotemporal feature modeling for wildlife detection and tracking in video data. The results underscored the significance of temporal modeling in achieving stable and reliable wildlife detection in dynamic environments.

Zhang et al. proposed ST-YOLO, a spatiotemporal enhanced YOLO framework for robust wildlife detection in videos. Their approach incorporated temporal feature aggregation to improve prediction consistency. The results indicated better stability when operating in changing or complex environments.

Rahman and colleagues designed a real-time system YOLOv11-based animal intrusion detection system with temporal smoothing and edge deployment. The temporal smoothing mechanism reduced false positives in video streams. Their work closely aligns with spatiotemporal detection enhancements.

Liu et al. introduced TCA-YOLO, a temporal consistency-aware YOLO framework for stable surveillance detection. The model explicitly enforced inter-frame consistency to reduce flickering detections. Experimental results demonstrated improved robustness in video-based monitoring.

From the above literature, it is evident that significant progress has been made in wildlife and

agricultural monitoring using YOLO-based object detection models, lightweight drone systems, IoT-integrated surveillance, and small-object enhancement techniques. Several studies have incorporated spatiotemporal modeling, temporal smoothing, and attention mechanisms to improve detection robustness in video streams. However, many existing approaches either focus primarily on small-object optimization, drone imagery, or IoT alert frameworks, with limited emphasis on maintaining stable real-time detection performance under dynamic farm environments affected by motion blur, illumination variation, and background clutter. Therefore, there remains a need for an efficient spatiotemporal wildlife detection framework that enhances prediction stability and reduces false alarms while preserving real-time inference capability for practical edge deployment in livestock farm protection systems.

Table 2.1 Summary of Existing Research Works

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
1.	Abdul Basit Mughal et al. 2025 [1]	Deep Learning for Dynamic Wildlife Monitoring: A Real-Time Approach	Real-time detection with interpretability	YOLOv11 + Grad-CAM	Forest wildlife dataset (multiple species, complex scenes)	mAP: 97.25%, F1-score: 95.65%, Accuracy: 95.14% (real-time capable)	Performance degrades under extreme occlusions and lighting
2.	Namratha et al. 2025 [2]	Automated approach for wildlife detection and tracking using YOLOv8 deep learning algorithm	Continuous identification and monitoring of wild animals through sequential video frame analysis.	YOLOv8	Custom dataset ~9000 annotated images	Accuracy: 89.26%, Precision: 83.60%	Lower performance under heavy occlusion and extremely low light; significant resource needs for remote deployment.
3.	Delwar et al. 2025 [3]	Real-Time Farm Surveillance Using IoT and YOLOv8	IoT-based surveillance integrated with deep learning	YOLOv8 + ESP32-CAM	580 images + real farm images	Accuracy: 99%, Precision: 99%	Limited night & weather evaluation

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
4.	Aditya Joshi et al. 2025 [4]	WBD-YOLO-AM: YOLOv8 with attention module and IoT-based wild boar detection and deterrence system for safeguarding small millets	Attention-enhanced detection with IoT-based alert and deterrence	YOLOv8 + Attention Module	Custom agricultural dataset of wild boar intrusion images/frames collected near small millet cropsto evaluate detection and real-time deterrence	mAP: 98.2%, Precision: 97.1%, Recall: 96.2%	Focused specifically on wild boar attacks in agriculture; might not generalize to other species; hardware reliability in extreme environments needs validation
5.	Zhibin Ma et al. 2025 [5]	Wildlife Real-Time Detection in Complex Forest Scenes Based on YOLOv5s Deep Learning Network	Lightweight detection with attention in forest environments	YOLOv5s + CBAM	Custom Houbara bird dataset (~40,000 annotated images)	mAP50: 96.27%, mAP75: 77.31%, mAP95: 71.78%; real-time detection ~43.7 ms/frame	Mostly applied to bird species; segmentation adds complexity

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
6.	Lyes Saad Saoud et al. 2025 [6]	Real-Time Threaded Houbara Detection and Segmentation for Wildlife Conservation using Mobile Platforms	Detection and segmentation on mobile platforms	YOLOv10 + Mobile SAM	Forest wildlife dataset (multiple species, complex scenes)	mAP: 97.25%, F1-score: 95.65%, Accuracy: 95.14% (real-time capable)	Performance degrades under extreme occlusions and lighting
7.	Amonga et al. 2025 [7]	Comparative Wildlife Detection Using CNNs	Comparative evaluation of CNN architectures	ResNet-101, Inception-V3	Wildlife benchmark dataset	Accuracy: 95%	Not real-time
8.	IJIRT Authors 2025 [8]	YOLOv8x IoT-Based Animal Intrusion Alert System	Intrusion detection with alert system	YOLOv8x + IoT alerts	Animal intrusion dataset	mAP: 94.3%	Precision–recall trade-off
9.	Zhu et al. 2025 [9]	YOLO-Wild ASM: Protected Wildlife Detection	Feature fusion with attention mechanism	YOLO + BiFPN + Attention	8000 wildlife images	mAP: 94.1%	Increased computation
10.	Thomas et al. 2025 [10]	Wildlife Detection Using YOLOv8	Image-based wildlife detection	YOLOv8	Wildlife image dataset	Accuracy: 93%	Limited dataset size

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
11.	Mallikarjuna et al. 2025 [11]	YOLOv8 Based Animal Intrusion Detection for Crops	Crop-field intrusion monitoring	YOLOv8	Custom farm intrusion dataset	Accuracy: 92.1%, Precision: 91.8%	Misses small/distant animals
12.	Shahrir et al. 2025 [12]	Hybrid YOLOv5-SSD IoT Animal Detection System	Hybrid detection with IoT alerts	YOLOv5 + SSD	Crop-field dataset	Elephant: 90% Boar: 85%	Night-time false alarms
13.	Shiney Thomas 2025 [13]	Wildlife Detection and Recognition using YOLOv8	A YOLOv8-driven deep learning approach for the simultaneous detection and classification of wild animals from both static images and live video streams.	YOLOv8	Dataset of wildlife video/image frames	Accuracy: 97	Limited dataset scope & generalization
14.	Nagaraj et al. 2025 [14]	Automated Wildlife Detection & Tracking using YOLOv8	Automated wildlife detection and tracking system using YOLOv8 to process real-time video streams	YOLOv8	9000 wildlife images	Accuracy: 89.26%, Precision: 83.6%	Low-light performance issues

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
15.	Qian et al. 2025 [15]	Real-Time Detection of Fish Feeding Behavior with PM-YOLO (YOLOv8 variant)	Behavior detection in videos	PM-YOLO (YOLOv8 variant)	Fish feeding behavior video dataset	mAP: 83.5%	Domain is aquatic species, but methodology relates to real-time detection
16.	Zhang, Y. et al., 2025 [16]	ST-YOLO: Spatio-Temporal Enhanced YOLO Framework for Robust Wildlife Detection in Video Streams	Spatio-temporal feature integration for stable wildlife detection in video streams	ST-YOLO (YOLO + Temporal Module)	Wildlife video dataset (multi-species outdoor scenes)	mAP: 96.8%, F1-score: 94.9%, Improved temporal stability	Increased computational cost due to temporal modeling
17.	Rahman, M. T. et al., 2025 [17]	Real-Time Animal Intrusion Detection Using YOLOv11 with Temporal Smoothing and Edge Deployment	Real-time intrusion detection with temporal smoothing and edge deployment	YOLOv11 + Temporal Smoothing	Farm surveillance dataset (real-time camera feeds)	Accuracy: 97.6%, Precision: 95.4%, Reduced false alarms	Performance affected on low-power edge devices

S. No	Author Name, year of publication	Title	Methodology	Algorithms used	Dataset Name (if any)	Result	Limitation
18.	Liu, X. et al., 2025 [18]	TCA-YOLO: Temporal Consistency Aware YOLO for Stable Object Detection in Surveillance Videos	Temporal consistency module integrated with YOLO for stable detection	TCA-YOLO (YOLO + Consistency Module)	Public surveillance video dataset	mAP@50: 95.9%, Recall: 96.2%, Significant flicker reduction	Slight latency increases due to frame buffering
19.	Sharma, P., & Verma, R., 2025 [19]	Intelligent Wildlife Monitoring System Using Deep Learning and IoT-Based Alert Mechanisms	Deep learning-based detection integrated with IoT alert system	YOLOv8 + IoT Notification System	Wildlife farm dataset (~5000 images)	Accuracy: 94.7%, Real-time alert generation	Limited performance under extreme weather conditions
20.	Chen, L. et al., 2025 [20]	Video-Based Wildlife Detection Using Hybrid YOLO and Attention Mechanisms for Real-Time Farm Protection	Hybrid attention-enhanced YOLO model for video-based detection	YOLO + Attention Mechanism	Custom wildlife dataset (multi-class animal data)	mAP: 97.1%, Precision: 96.3%, Enhanced small object detection	Requires large annotated dataset for optimal performance

2.3 DISADVANTAGES OF THE EXISTING SYSTEM

Existing wildlife detection systems that rely solely on frame-by-frame object detection, without incorporating temporal consistency mechanisms, face several significant challenges when deployed in real-world video surveillance environments. In such systems, each video frame is processed independently, meaning that the detection model makes predictions without considering information from previous or subsequent frames. As a result, these systems frequently suffer from detection flickering, where an animal is detected in one frame but missed in the next, even though the animal remains present in the scene. This variation decreases the reliability of the monitoring system and restricts its effectiveness in applications that require continuous and stable tracking, such as livestock farm protection or wildlife corridor observation.

Another important limitation of frame-based detection systems is the increased occurrence of false positives and unstable bounding box predictions. Because the model analyses each frame in isolation, it cannot verify whether a detected object actually persists over time. This often leads to incorrect detections caused by environmental factors such as moving vegetation, shadows, camera noise, or sudden lighting changes. Furthermore, these systems struggle when animals move quickly, become partially occluded, or appear blurred due to motion. These limitations demonstrate the importance of integrating temporal consistency mechanisms to achieve more stable and reliable wildlife detection in real-time video monitoring systems.

2.4 PROPOSED SYSTEM

Proposed system uses YOLOv11 to detect harmful wild animals, applying temporal consistency and email alerts for reliable real-time intrusion monitoring. The system utilizes augmented datasets, minimizes incorrect detections, and offers an easy-to-use interface that supports efficient wildlife monitoring while helping prevent animal intrusions.

2.4.1 SUMMARY OF THE DATASET

The data that has been used for this research is obtained from Kaggle, and it consists of images of 13 dangerous wild animal species that are often involved in the destruction of crops and intrusion of livestock in rural and adjacent forest areas. Initially, the dataset exhibited significant class imbalance, where certain animal categories had substantially more samples than others, potentially leading to biased learning and reduced generalization performance. To address this limitation, several data augmentation techniques were applied, including image flipping, rotation, cropping, and the addition of noise to improve dataset diversity. These transformations enhanced dataset diversity and improved

robustness against real-world challenges such as occlusion, illumination variation, motion artifacts, and background clutter. After augmentation, the dataset size increased to over 5,500 images, resulting in improved class distribution and better environmental representation.

During the annotation stage, the dataset initially contained bounding box labels expressed in absolute pixel coordinates, specified through class names along with xmin, ymin, xmax, and ymax values. As the YOLO architecture requires normalized annotations, these labels were reformatted into the YOLO representation. In this format, each object is encoded using a class index, the normalized center position (x_center , y_center), along with the relative width and height of each bounding box.

The transformation involved calculating the midpoint of each bounding box and its dimensions with respect to the total image width and height, followed by scaling the values to fall within the range of 0 to 1. Adopting this unified labeling scheme ensured seamless compatibility with the Ultralytics YOLO framework and supported efficient training of the detection model.

2.4.2 PROPOSED ARCHITECTURE DIAGRAM

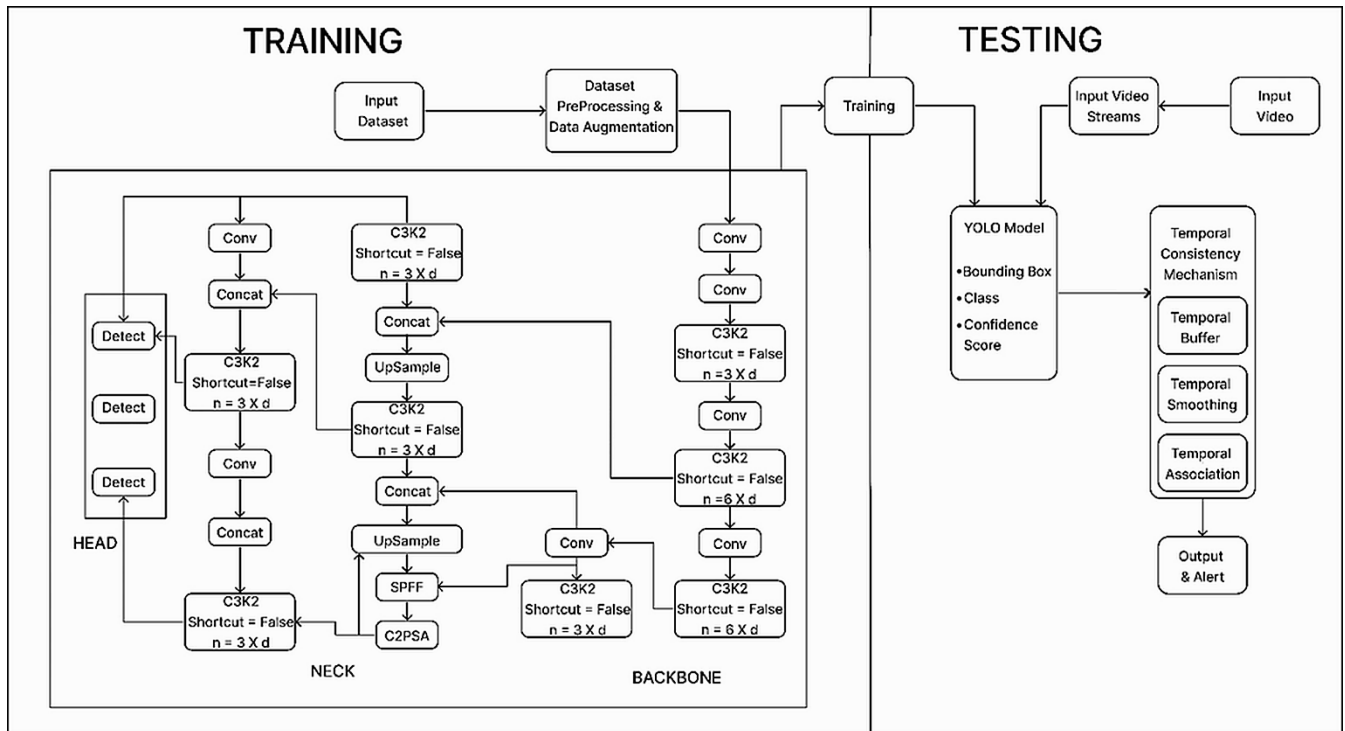


Fig 2.1 Proposed Architecture for Harmful Wild Animal Detection using YOLOV11

2.4.3 PROPOSED ALGORITHM

Step 1: Start

Step 2: Collect the wildlife image dataset containing 13 harmful wild animal classes from Kaggle.

Step 3: Convert the annotation format by transforming Pascal VOC XML labels into YOLO TXT format with normalized bounding box values.

Step 4: Organize the data by placing images and their corresponding labels into separate folders for training, validation, and testing.

Step 5: Data Preprocessing:

- Step 5.1: Adjust all images so they match the required input dimensions, for example 640×640 pixels.
- Step 5.2: Normalize pixel values for consistent scaling.
- Step 5.3: Apply data augmentation (flip, brightness change, rotation, scaling).
- Step 5.4: Balance the dataset to reduce class imbalance (final dataset: 4500+ images across 13 classes).

Step 6: Model Development (YOLOv11):

- Step 6.1: Configure training parameters (learning rate, batch size, epochs, image size).
- Step 6.2: Train the YOLOv11 object detection model.
- Step 6.3: Monitor training using Box Loss, Classification Loss, and mAP.
- Step 6.4: Save the best-performing model weights as best.pt.

Step 7: Temporal Consistency Integration:

- Step 7.1: Track detections across consecutive video frames.
- Step 7.2: Confirm detection only after N continuous frames.
- Step 7.3: Reduce false detections caused by single-frame errors.

Step 8:

- Step 8.1: Test the trained model on the test dataset.
- Step 8.2: Calculate Precision, Recall, F1-Score, mAP@50, and mAP@50–95.
- Step 8.3: Compare standard YOLO detection with temporally filtered detection.

Step 9: Real-Time Alert System:

- Step 9.1: Detect harmful animal intrusion in video streams.
- Step 9.2: Trigger an email alert when intrusion is confirmed.
- Step 9.3: Attach the detected frame image as evidence.

Step 10: User Interface Implementation:

- Step 10.1: Develop an interface to upload video files.
- Step 10.2: Display real-time detection results with bounding boxes.
- Step 10.3: Show alert notifications within the interface.
- Step 10.4: Connect the interface with the trained model and email system.

Step 11: Stop

2.5 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed wildlife monitoring framework offers several practical benefits for agricultural lands and residential areas situated close to forest regions. It provides real-time monitoring through the YOLOv11 detection framework, which can rapidly and accurately recognize various wild animals that may threaten crops, livestock, or human safety. Early detection plays an important role in preventing damage, as farmers can respond immediately when an animal enters a monitored area. Instead of depending only on single-frame predictions, the system confirms whether the detected animal continues to appear over time. This method significantly reduces false alarms that may occur due to moving leaves, shadows, or sudden environmental changes, this helps enhance the overall consistency and dependability of the monitoring system.

Another important component of the system is the automated email alert feature, which sends instant notifications to farm owners or responsible authorities whenever wildlife is detected near the monitored location. This rapid communication allows users to take timely action and helps minimize potential damage to crops or property. The system is also designed to recognize multiple animal categories, making it adaptable for different geographic regions where various species may be present. To enhance detection performance in varying environmental conditions, the dataset is first processed and augmented using methods like brightness modification, scaling, and other image transformations. These techniques enable the model to perform reliably even in low-light situations or during changing weather conditions. Additionally, the system features an intuitive interface through which users can submit and examine video inputs without needing specialized technical expertise, making it a practical and economical approach to help reduce human–wildlife conflicts.

CHAPTER-3

ANALYSIS

CHAPTER-3

ANALYSIS

3.1 INTRODUCTION

The developed solution is a real-time wildlife intrusion monitoring framework created to safeguard livestock farms in rural and forest-border areas. It combines the YOLOv11 object detection model with a temporal consistency mechanism, which enhances the stability of detections when processing continuous video streams. It effectively addresses challenges such as motion blur, scale variation, partial occlusion, illumination fluctuations, and background clutter that commonly occur in outdoor farm surveillance environments.

Although the system achieves high detection accuracy and robustness, certain limitations exist. The model may occasionally misclassify visually similar animal species and its performance decreases under extreme low-light or near-dark conditions. Furthermore, the deployment in real-time requires sufficient computational power, particularly in handling high-definition videos. However, the developed framework provides a viable solution for the monitoring of wildlife and the security of livestock despite the challenges.

3.2 SOFTWARE REQUIREMENT SPECIFICATION

The developed wildlife detection system uses computer vision and deep learning techniques to identify and monitor the presence of dangerous wild animals. In this case, the developed system uses the YOLOv11 model to analyse the video frames and identify the objects. In addition, the system applies temporal smoothing to stabilize the detection results. It is designed for deployment in farm environments using edge-based devices and real-time surveillance cameras.

3.2.1 FUNCTIONAL REQUIREMENTS

The functional requirements of the wildlife intrusion detection system include:

1. Dataset Collection:
Collect images of 13 harmful wild animal species from Kaggle and other public datasets.
2. Data Augmentation:
Apply augmentation techniques such as horizontal flip, vertical flip, rotation, blur, scaling, cropping, brightness adjustment, and noise injection to balance the dataset.
3. Annotation Conversion:
Convert the bounding box annotations from the custom coordinate format into the YOLO

normalized format consisting of class ID, centre coordinates, width, and height.

4. Model Development:

Implement YOLOv11 for multi-class object detection.

5. Temporal Consistency Integration:

Apply inter-frame confidence aggregation and bounding box smoothing to reduce flickering and false positives.

6. Training Phase:

During this stage, the YOLOv11 network is trained with the augmented dataset while adjusting the hyperparameters to achieve better detection performance.

7. Evaluation of the Model:

Metrics to evaluate the performance of the model are precision, recall, F1-score, mAP@50, and mAP@50–95.

8. Real-Time Detection:

Perform frame-level detection on live or recorded surveillance video.

9. Alert Generation:

Generate alerts when harmful animals are detected in farm premises.

10. Edge Deployment:

Optimize the model for deployment on GPU-enabled edge devices.

3.2.2 NON-FUNCTIONAL REQUIREMENTS

Front-End: Streamlit (Optional Dashboard)

Streamlit is used to build an interactive web-based interface for displaying detection results. It enables real-time visualization of bounding boxes, prediction confidence scores, and alert notifications. The interface ensures usability, responsiveness, and easy navigation for farm owners or monitoring authorities. The framework is built to ensure scalability, reliability, and security. The system also allows future enhancements, including animal tracking capabilities and integration with IoT technologies. The user interface is lightweight and requires minimal technical knowledge to operate.

Back-End: Python with YOLOv11 (Ultralytics Framework)

The backend is implemented using Python and the Ultralytics YOLOv11 framework. The system processes video frames, performs object detection, and applies temporal smoothing techniques to enhance prediction stability. The backend supports GPU acceleration for faster inference.

The framework ensures efficient data handling, real-time detection, and reproducibility. The integration of temporal consistency improves robustness without modifying the core detection architecture.

3.2.3 HARDWARE REQUIREMENTS

System: Intel i5 or above

RAM: 8 GB or above

GPU: RTX 2050 / RTX 3050 or higher

Storage: 256 GB SSD or above

Input Device: Keyboard and Mouse

Output Device: Monitor / Laptop

3.2.4 SOFTWARE REQUIREMENTS

Operating System: Windows 10/11 (64-bit)

Programming Language: Python 3.10 or above

Development Platform: Jupyter Notebook / VS Code

Browser: Google Chrome

3.2.5 PACKAGE REQUIREMENTS

Ultralytics (YOLOv11):

Ultralytics YOLOv11 is a deep learning framework developed to perform real-time object detection and support different computer vision tasks. It offers a simplified and efficient processing workflow for training, validation, and deployment of YOLO models.

- Pre-trained models for fast object detection and transfer learning
- End-to-end training pipeline with dataset configuration and augmentation
- Facilitates fast real-time detection with bounding box information, category labels, and prediction confidence values.
- The framework includes native support for computing standard detection evaluation metrics, namely precision, recall, and mean average precision, without requiring external libraries.

TensorFlow / PyTorch:

TensorFlow and PyTorch are widely used platforms for developing, training, and deploying neural network models. They provide efficient tools for building complex machine learning architectures.

- GPU and TPU acceleration for faster model training and computation
- Automatic differentiation for optimizing neural network parameters
- Support for building convolutional neural networks (CNNs), RNNs, and transformer models

- Extensive libraries for data processing, model evaluation, and deployment
- Integration with computer vision libraries such as OpenCV for preprocessing tasks

OpenCV:

In this project, OpenCV was utilized to handle frame-level video processing operations, including reading input streams, resizing frames to the required dimensions, and performing colour space conversions prior to feeding data into the detection model.

It supports:

- Image loading and manipulation (resizing, cropping, color conversion)
- Feature extraction (contours, edges, shapes)
- Object detection and tracking
- Face recognition and segmentation

OpenCV integrates with TensorFlow and Keras, allowing for image preprocessing before feeding data into deep learning models.

NumPy:

NumPy (Numerical Python) is the core library for numerical computing in Python. It offers support for:

- Handling multi-dimensional arrays. (ndarray)
- Mathematical computations such as linear algebra operations and Fourier transforms.
- Element-wise operations and broadcasting
- Efficient memory management for processing large datasets.

Within this system, NumPy was employed to manage numerical operations on image arrays and bounding box coordinates, enabling efficient manipulation of large multi-dimensional data structures throughout the preprocessing and inference pipeline.

Pandas:

Pandas is a Python library used for data analysis and manipulation, enabling efficient handling of structured data through Data Frame and Series objects. It enables:

- Data cleaning (handling missing values, duplicates)
- Data transformation (merging, grouping, reshaping)
- Importing and exporting data from CSV, Excel, JSON, and databases

- Time series analysis and handling of categorical data
- Pandas simplifies data preprocessing and feature preparation for machine learning applications.

Matplotlib:

It is used to display the performance of the model while training with the help of precision, recall, F1-score, and mAP.

- A plotting library used for creating 2D plots and graphs, including line plots, bar plots, and scatter plots. It allows full control over plot elements (labels, colors, gridlines).

Scikit-learn:

In this project, Scikit-learn was incorporated during the evaluation phase to compute per-class classification metrics including precision, recall, and F1-score, supplementing the standard YOLO evaluation outputs with a more detailed performance breakdown across all thirteen animal categories.

It includes:

- Data preprocessing utilities such as scaling and normalization
- Model selection tools including cross-validation and hyperparameter optimization.
- Algorithms: Decision trees, Support Vector Machine (SVM), Random Forest, Gradient Boosting.
- Metrics: Accuracy, Precision, Recall, F1-score.
- Easy integration with deep learning frameworks like TensorFlow for building advanced machine learning systems

OS & Glob:

The OS and Glob libraries are used for file system operations and dataset management. They help in accessing directories, organizing files, and loading large collections of images for machine learning workflows.

- Directory and file management such as creating, renaming, and deleting folders
- Navigating file paths and accessing dataset locations
- Automatically retrieving files using pattern matching (e.g., loading all .jpg or .png images)
- Efficient dataset loading for training and testing pipelines
- Simplifies batch processing of images stored in multiple folders

CHAPTER-4

DESIGN

CHAPTER-4

DESIGN

4.1 INTRODUCTION

Designing an intelligent system for harmful wild animal detection using deep learning involves a structured workflow that integrates dataset preparation, model development, validation, and deployment. The main goal of this system is to automatically identify wild animals that could threaten livestock and agricultural areas. The framework employs the YOLOv11 object detection model along with a temporal consistency mechanism to enhance the stability of detections when processing video streams. The initial stage of the design process involves collecting a diverse dataset of wildlife images containing multiple animal species commonly responsible for crop damage and livestock attacks. The dataset consists of images in JPEG and PNG formats and includes various animal classes such as bear, cheetah, crocodile, eagle, elephant, fox, leopard, lion, monkey, pig, scorpion, snake, and tiger. These images cover various environmental conditions, camera viewpoints, and backgrounds to support effective and reliable model training.

After obtaining the dataset, several preprocessing and preparation steps are performed to maintain the quality and usability of the training information. The initial annotations in the dataset store bounding box coordinates in absolute pixel values. All bounding box annotations were transformed into the YOLO-compatible normalized format, where each entry consists of a class index along with the centre coordinates, width, and height of the corresponding bounding box, all expressed as fractions of the image dimensions. Prior to training, every image was rescaled to match the input resolution expected by the YOLOv11 architecture. To strengthen the model's ability to generalize across varied real-world conditions, a range of augmentation strategies were applied to the training data, including horizontal and vertical flipping, random rotation, brightness and contrast variation, blur effects, scaling, random cropping, and synthetic noise injection. These transformations increase dataset variation and help the model detect animals under different environmental conditions.

After preprocessing, the next stage focuses on feature extraction and model training using the YOLOv11 architecture. YOLOv11 functions as a real-time object detection architecture that integrates feature extraction, object localization, and classification within a unified neural network. The underlying architecture comprises three core components: a backbone responsible for extracting spatial representations from input images, a neck module that aggregates and fuses features across multiple scales, and a detection head that jointly predicts bounding box coordinates and class probabilities. For training purposes, the collected dataset was partitioned into three subsets, allocating 70% for training, 15% for validation, and the remaining 15% for testing, ensuring unbiased performance assessment and

mitigating the risk of overfitting. Through exposure to labelled training samples, the model progressively learns to associate distinctive visual cues such as body shapes, surface textures, and species-specific characteristics with their corresponding animal categories.

To further enhance detection reliability in video-based wildlife monitoring scenarios, a temporal consistency module is integrated with the YOLOv11 detection pipeline. Traditional object detection models perform predictions independently for each frame, which may lead to unstable detections or flickering bounding boxes when objects are partially occluded or briefly disappear. The temporal consistency mechanism addresses this limitation by aggregating detection results across multiple consecutive frames and validating consistent predictions over time. The method improves the stability of detection and reduces the false alarm rate of the real-time monitoring system. After the model attains the desired validation results, it is then tested using new images and video frames to check its ability to generalize.

To evaluate the performance, metrics such as precision, recall, F1-score, and mAP are used. These metrics help to evaluate the accuracy and reliability of the detections. The objective is to obtain strong detection performance while keeping the inference process fast enough for real-time use.

Lastly, the trained model is integrated into a monitoring system to monitor video streams in search of wildlife intrusion. The system highlights detected animals using bounding boxes and class labels, enabling farmers or authorities to quickly identify potential threats. Continuous monitoring and future model improvements, such as incorporating additional wildlife classes and improving low-light detection, can further enhance the system's effectiveness in preventing human-wildlife conflicts and protecting livestock and agricultural resources.

4.2 UML DIAGRAMS

4.2.1 STATE CHART DIAGRAM

The state chart diagram for the proposed Harmful Wild Animal Detection System using YOLOv11 with Temporal Consistency represents the sequence of operations involved in detecting wildlife intrusion and protecting livestock farms. The process begins with Dataset Collection, where images of harmful wild animals such as bear, cheetah, crocodile, eagle, elephant, fox, leopard, lion, monkey, pig, scorpion, snake, and tiger are collected from publicly available datasets. These images contain different backgrounds, environmental conditions, and camera angles to ensure diversity and robustness during model training.

The next stage is Dataset Cleaning, where irrelevant classes and duplicate or low-quality images are removed from the dataset. Dataset balancing techniques are also applied to ensure that all animal classes are fairly represented, which helps prevent model bias during training.

After cleaning the dataset, the YOLO format conversion and normalization step transforms the original bounding box annotations into the YOLO labelling structure. During this step, the bounding box coordinates are transformed into a normalized format that includes the class ID, centre position (x_center , y_center), and the relative width and height based on the image dimensions.

After completing the dataset preparation stage, the workflow moves to training the YOLOv11 model. During this stage, the preprocessed dataset is used to train the YOLOv11 neural network to detect and classify harmful wildlife species. The model learns spatial features such as shapes, textures, and visual patterns that distinguish different animal classes.

Once the model is trained, a Temporal Consistency Integration mechanism is applied. Traditional object detection models process video frames independently, which may result in unstable detections or flickering bounding boxes. The temporal consistency mechanism examines detection outputs across successive frames to maintain stable predictions, reduce false alarms, and enhance detection accuracy in real-time video streams.

After integrating temporal consistency, the system proceeds to the Model Testing User Interface stage. In this stage, the trained model undergoes testing through a video-based interface where users can upload or stream video footage. The system processes each frame and displays detected animals using bounding boxes and class labels.

Finally, the system enters the deployment stage, where the trained and validated model is implemented as a real-time livestock protection system. It continuously monitors surveillance camera video streams to detect harmful wildlife in farm environments and generate alerts when animals are detected. The deployment process is illustrated in Fig. 4.1.

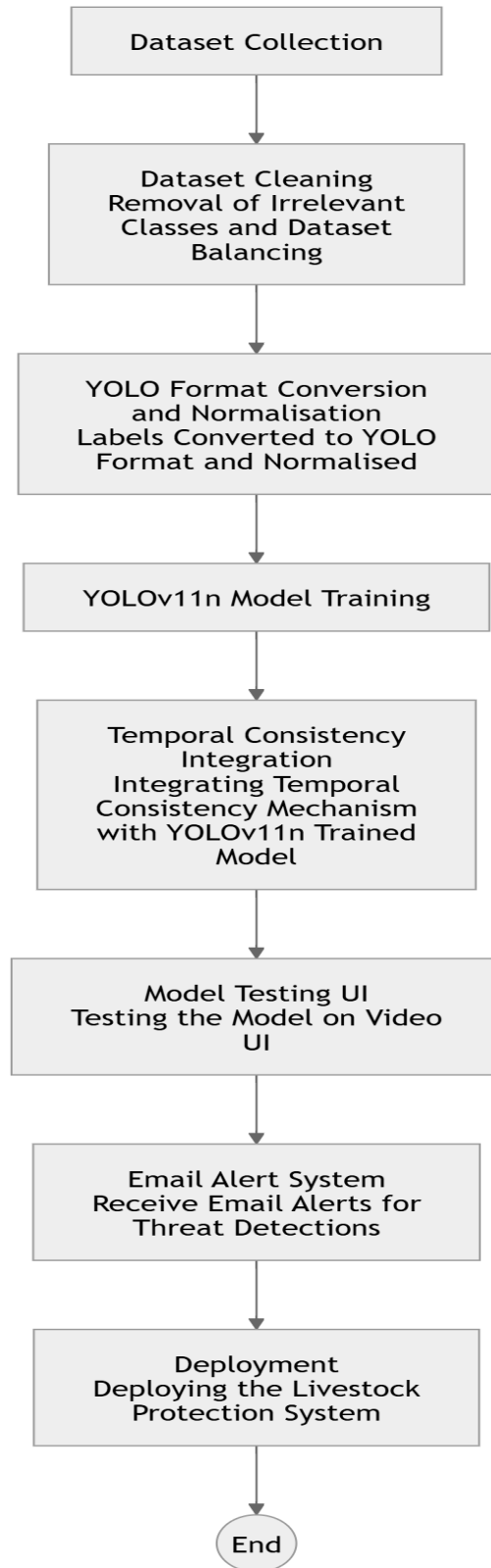


Fig 4.1 State Chart Diagram

4.2.2 SEQUENCE DIAGRAM

The sequence diagram illustrates the interaction flow of the proposed livestock farm protection system developed to detect harmful wild animals through automated video analysis. The process begins when the user uploads a surveillance video of the livestock farm using the video upload interface. Once the video is successfully submitted, the system transfers the video file to the processing module for further analysis. During the preprocessing stage, the system extracts individual frames from the uploaded video and prepares them for object detection. These frames are formatted and optimized so that they can be efficiently processed by the detection model. Each extracted frame is then provided as input to the trained YOLOv11 model, which analyses the visual content of the frame. The model carries out object detection to recognize wild animals such as tigers, lions, elephants, or snakes, producing detection outputs that include object bounding boxes, predicted class labels, and associated confidence values.

After the initial detection stage, the system applies a temporal consistency mechanism to enhance the reliability of predictions. This mechanism compares detection results across consecutive frames to confirm whether the detected animal remains visible over time. By analysing multiple frames, the system removes unstable detections caused by background movement, lighting variations, or temporary noise. Once the verification process is completed, the system generates the final detection results. If a harmful animal is confirmed, the alert module automatically sends an email notification to the farmer or system administrator. The system also displays the detected frames with labelled bounding boxes on the user interface for visual confirmation. The sequence diagram representing this interaction flow is illustrated in Fig. 4.2.

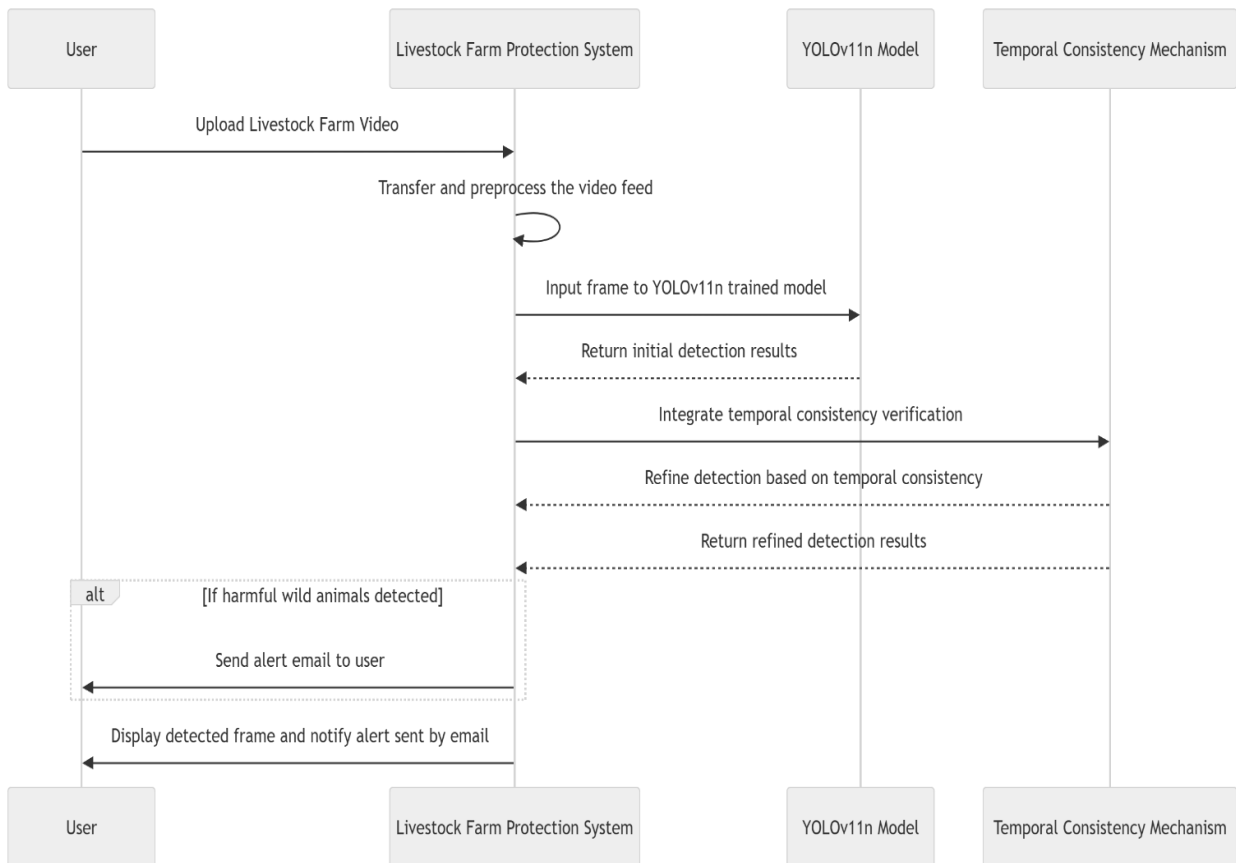


Fig 4.2 Sequence Diagram

4.2.3 USE CASE DIAGRAM

The use case diagram for the proposed Wild Animal Detection System for Livestock Farm Protection illustrates the interactions between the user and the system components involved in detecting wildlife near farm areas. The process begins when the user uploads an image or video captured from surveillance cameras installed around the livestock farm. After the media is uploaded, the system displays the input to ensure that the correct file has been selected for analysis. The system then processes the media using the YOLOv11 object detection model, which identifies potential animals present in the frames. An object tracking module is also incorporated to maintain continuity of detected objects across consecutive frames, improving detection stability. Additionally, a confidence score calculation mechanism evaluates the reliability of each detection before confirming the result. Based on this analysis, the system generates detection outputs and identifies the detected animal species. If a harmful wild animal is detected, the system highlights it using bounding boxes along with prediction confidence scores. The annotated results are then displayed to the user for visual verification. The use case diagram representing this interaction is illustrated in Fig. 4.3.

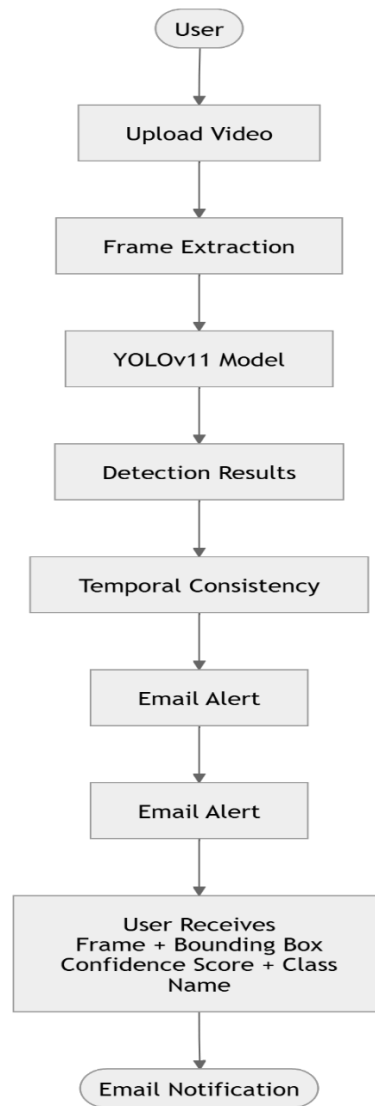


Fig 4.3 Use Case Diagram

4.2.4 COLLABORATIVE DIAGRAM

The collaborative diagram of the proposed Wild Animal Detection System for Livestock Farm Protection illustrates how different system components work together to process surveillance video data and detect harmful wildlife. The process begins when the user uploads a livestock farm monitoring video through the web-based user interface. After the video is uploaded, the system transfers the video file to the processing module. The frame extraction module then divides the uploaded video into multiple frames so that each frame can be analysed individually. These frames serve as input to the YOLOv11 object detection model. The model examines each frame and identifies objects that may represent animals in the scene. During this stage, the model produces detection results that include bounding boxes, predicted class categories, and confidence values for the identified animals.

After the initial detection step, the results are forwarded to the temporal consistency module to improve detection stability across consecutive frames. This module performs operations such as tracking detected objects, smoothing predictions, and confirming detections that remain consistent over time. These steps help reduce false positives caused by environmental changes or background movement. Once the detection is verified, the system checks whether the detected object corresponds to a harmful wild animal. If confirmed, the email alert system sends a notification containing the detected frame, bounding box, class name, and confidence score. The system also displays the detection results on the user interface, as illustrated in Fig. 4.4.

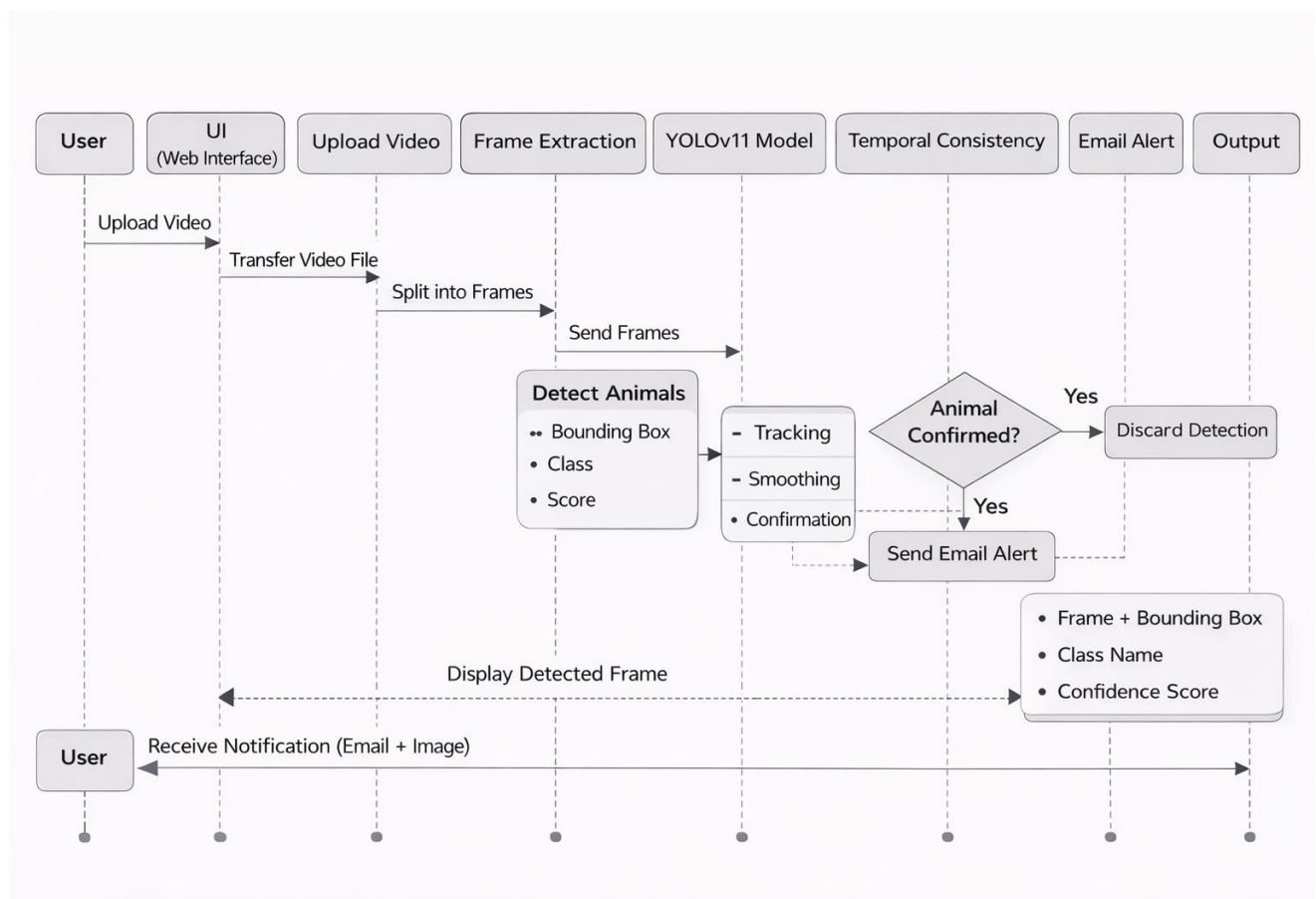


Fig 4.4 Collaborative Diagram

4.3 MODULE DESIGN AND ORGANISATION

The entire module is constructed in 8 modules:

1. Data Collection
2. Data Integration
3. Data Preprocessing
4. Model Training
5. Model Testing
6. Temporal Consistency Integration
7. Alert and Notification Module
8. User Interface Module

4.4 SUMMARY

The proposed Wild Animal Detection System for Livestock Farm Protection is designed as an intelligent deep learning-based monitoring system that integrates video processing, object detection, temporal verification, and automated alert mechanisms. The system utilizes the YOLOv11 object detection model combined with a temporal consistency mechanism to accurately detect harmful wild animals in surveillance footage while reducing false detections.

The system architecture follows a structured modular design, ensuring scalability, maintainability, and efficient real-time performance. Various UML diagrams such as state diagrams, sequence diagrams, use case diagrams, and collaborative diagrams are used to represent system workflows, component interactions, and data flow.

This modular and UML-based design provides a clear understanding of the system structure and ensures seamless interaction between the user interface, detection model, temporal verification module, and alert system. The developed framework therefore presents a practical and dependable approach for continuous livestock farm surveillance, enabling early identification of wildlife intrusions and empowering farm operators to initiate preventive responses before significant damage occurs.

CHAPTER-5

IMPLEMENTATION AND RESULTS

CHAPTER-5

IMPLEMENTATION AND RESULTS

5.1 INTRODUCTION TO SYSTEM IMPLEMENTATION

Due to recent progress in artificial intelligence and computer vision, automated monitoring systems are being widely implemented to tackle practical problems like wildlife intrusion in livestock farms. Farms located near forest regions often experience crop damage, livestock attacks, and safety risks due to the presence of harmful wild animals. Manual monitoring approaches have proven inadequate in terms of response speed and operational efficiency, reinforcing the necessity for automated intelligent systems capable of delivering real-time detection and instant alerts. This project aims to develop an automated framework for detecting wildlife intrusions using the YOLOv11 deep learning model combined with a temporal consistency technique to improve detection stability in continuous video streams.

The proposed system employs a trained deep learning object detection model to identify various harmful wild animal species from live surveillance footage. To further enhance detection reliability in video surveillance scenarios, a temporal consistency mechanism is integrated to reduce prediction flickering and false positives by enforcing inter-frame coherence across consecutive frames.

The dataset used for this study contains images of 13 harmful wild animal classes collected from Kaggle. To address class imbalance in the dataset, augmentation methods including flipping, rotation, blurring, brightness variation, scaling, cropping, and noise injection were applied to diversify the training samples. After these augmentations, the dataset size grew to more than 5,500 images. The training process was carried out using optimized parameters including appropriate batch size, image resolution, and training epochs to achieve improved detection accuracy.

The trained model was assessed using standard detection metrics including precision, recall, F1-score, mAP@50, and mAP@50–95. The model attained a precision of 0.9843 and a recall of 0.9691, yielding an F1-score of 0.9766, which reflects accurate animal identification with very few false detections. Furthermore, mAP@50 and mAP@50–95 scores of 0.9934 and 0.9658 respectively confirm robust detection performance across varying IoU thresholds, validating the overall effectiveness of the proposed system.

This chapter describes the full implementation procedure of the developed system, covering dataset preparation, preprocessing steps, model training, validation, testing, and the incorporation of the temporal consistency mechanism. It also outlines the experimental configuration employed for

system evaluation and examines the results obtained during testing. The performance analysis verifies the effectiveness of the wildlife intrusion detection framework for real-time livestock farm surveillance and protection.

5.2 PREPARATION AND PREPROCESSING OF THE DATASET

- **Collection of Dataset:**

The dataset acquisition module involves gathering image data containing harmful wild animal species that commonly intrude into livestock farms and agricultural areas. The images were obtained from publicly accessible datasets available on Kaggle, containing 13 different classes of wild animals such as elephants, bears, tigers, pigs, and other species known to cause crop damage and livestock attacks. Initially, the dataset was imbalanced, meaning that some animal classes contained more samples than others. The collected images were saved in common formats such as JPEG and PNG and served as the main dataset for training the object detection model.

- **Data Augmentation and Dataset Balancing:**

To improve the model's generalization capability and address class imbalance, various augmentation techniques were applied to the dataset. These transformations increased the diversity of the training data and helped the model learn stronger visual features under different environmental conditions. The augmentation techniques include horizontal flipping, vertical flipping, random rotation, Gaussian blur, motion blur, brightness and contrast adjustment, random scaling, cropping, and noise injection. After applying these augmentation methods, the dataset size increased to more than 5,500 images, resulting in a more balanced dataset and improved representation of environmental conditions such as lighting variations, occlusion, and background complexity.

- **Annotation Conversion Module:**

The original dataset annotations were provided in a custom bounding box format where each label contained the class name and absolute pixel coordinates in the format:

class_name, xmin, ymin, xmax, ymax

Since the YOLOv11 framework requires a different annotation structure, the labels were converted into the YOLO format, which uses normalized coordinates relative to image

dimensions. In the YOLO annotation format, each object is described using the parameters class ID, center coordinates (x_center , y_center), width, and height.

Here, `class_id` represents the numeric index of the animal class, while the bounding box coordinates represent the normalized center position and dimensions of the object. This conversion enables efficient training and compatibility with the Ultralytics YOLOv11 framework.

- **Model Development Stage:**

The model development component focuses on implementing the YOLOv11 architecture for real-time object detection. YOLOv11 is a single-stage object detection model that simultaneously performs object localization and classification in a single forward pass. The architecture includes three main parts: a backbone network that extracts visual features, a neck module responsible for multi-scale feature integration, and a detection head that outputs bounding box predictions along with class probabilities. The model was initialized with pre-trained YOLOv11 weights and subsequently trained on the prepared dataset using optimized parameters, including batch size, image resolution, and the number of training epochs.

- **Model Training Module:**

The model training module focuses on training the YOLOv11 network with the augmented dataset. During training, images are resized to 640×640 pixels, with a batch size of 16, and the process runs for 100 epochs. The model is trained using GPU acceleration to improve computational efficiency. During training, the model learns spatial features of different animal classes and optimizes its parameters using loss functions that measure bounding box regression, classification accuracy, and objectness confidence. Validation is conducted at regular intervals to track the model's performance and reduce the risk of overfitting.

- **Performance Evaluation Module:**

The trained model was evaluated on a separate test set of over 1,000 annotated images using standard detection metrics. The results demonstrated strong performance, achieving a precision of 0.9843, recall of 0.9691, and F1-score of 0.9766. Additionally, mAP@50 and mAP@50–95 scores of 0.9934 and 0.9658 respectively confirm reliable detection across varying IoU thresholds, establishing the effectiveness of the proposed system in identifying harmful wildlife species under diverse environmental conditions.

- **Temporal Consistency Module:**

To improve detection stability in video streams, a temporal consistency mechanism is integrated with the YOLOv11 model. Traditional frame-by-frame detection often produces flickering predictions or false positives due to sudden background changes, motion blur, or temporary occlusions. The temporal consistency module addresses this issue by aggregating detection confidence across consecutive frames and enforcing inter-frame consistency. This approach stabilizes bounding box predictions and significantly reduces detection flickering without modifying the original YOLOv11 architecture.

- **Prediction and Video Detection Module:**

The prediction module processes images and video frames to identify the presence of harmful wild animals. Each video frame is processed by the trained YOLOv11 model, producing object bounding boxes, predicted class categories, and confidence values for the identified objects. When combined with a temporal consistency mechanism, the system confirms detections only when an object is consistently detected across multiple frames. This ensures more reliable predictions in real-time wildlife surveillance scenarios.

- **User Monitoring and Visualization Module:**

The system provides visual output by displaying bounding boxes around detected animals along with predicted class labels and corresponding confidence values. The detection outcomes can be viewed in real-time on video streams, making it easier for farm owners and monitoring authorities to identify wildlife intrusion events. This visualization capability allows the system to function as an intelligent surveillance tool for livestock farm protection.

In conclusion, the proposed wildlife intrusion detection system integrates dataset preparation, YOLOv11-based object detection, temporal consistency modeling, and real-time video monitoring to build an efficient AI-driven surveillance framework. By combining spatial detection with temporal stability, the system provides reliable and accurate detection of harmful wild animals, contributing to improved livestock protection and reduced human–wildlife conflict.

5.3 METHOD OF IMPLEMENTATION

Data balance.ipynb

```
import argparse
import uuid
import os
import random
import glob
import numpy as np
import cv2
import albumentations as A

def read_label_file(path):
    """
    Read label file lines like: "Bear x_min y_min x_max y_max"
    Returns list of (class_name, [x_min, y_min, x_max, y_max])
    """
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]
            try:
                coords = list(map(float, parts[1:5]))
            except Exception:
                continue
            boxes.append((cls, coords))
    return boxes
```

```

def write_label_file(path, boxes):
    # boxes: list of (class_name, [x_min, y_min, x_max, y_max])
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            # format floats with reasonable precision
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    # Look for .txt matching stem
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c

    # fallback: sometimes label names may differ; try any file that startswith stem
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def build_transform():
    # Compose a set of random augmentations that support pascal_voc bboxes
    return A.Compose(
        [
            A.OneOf([
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.1),
            ], p=0.5),
            A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.2, rotate_limit=20),
            A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
        ]
    )

```

```

        A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10),
        A.JpegCompression(quality_lower=75, quality_upper=95, p=0.3),
    ],
    bbox_params=A.BboxParams(format="pascal_voc", label_fields=["class_labels"]),
)

def main(images_dir, labels_dir, target_count=400, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    idx = 0
    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        src = random.choice(images)
        img = cv2.imread(src)
        if img is None:
            continue
        h, w = img.shape[:2]

        label_path = find_label_for_image(src, labels_dir)
        if label_path is None:
            # skip if no label (could be allowed if you want unlabeled augment)
            continue

        boxes_raw = read_label_file(label_path)
        if not boxes_raw:

```



```

        continue

bboxes = [coords for (cls, coords) in boxes_raw]
class_labels = [cls for (cls, coords) in boxes_raw]

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception as e:
    # transform may fail for some inputs; skip
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    # bbox vanished after augmentation; skip or retry
    continue

# create new filename
src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f'{src_stem}_aug_{uuid.uuid4().hex[:8]}'
# keep same extension as source
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

# save image
success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue

# clip boxes to image and ensure they are in [0,w]x[0,h]
clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    # discard degenerate boxes

```

```

        if x2 - x1 < 1 or y2 - y1 < 1:
            continue
        clipped_boxes.append((lbl, [x1, y1, x2, y2]))
    if not clipped_boxes:
        # nothing left, remove image and continue
        try:
            os.remove(new_image_path)
        except Exception:
            pass
        continue

    write_label_file(new_label_path, clipped_boxes)
    generated += 1
    if generated % 10 == 0 or generated == needed:
        print(f'Generated {generated}/{needed}')

print(f'Done: generated {generated} augmented images (attempts {attempts})')

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Bear\\Label",
help="Path to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)
def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue

```

```

        cls = parts[0]
        try:
            coords = list(map(float, parts[1:5]))
        except Exception:
            continue
        boxes.append((cls, coords))

    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def main(images_dir, labels_dir, target_count=400, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f"Found {existing} images in {images_dir}")
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

```

```

needed = target_count - existing
print(f"Need to generate {needed} augmented images")

transform = build_transform()

generated = 0
attempts = 0
# ensure output folder exists
os.makedirs(images_dir, exist_ok=True)

while generated < needed and attempts < needed * max_tries:
    attempts += 1
    if not images:
        print("No source images available to augment. Aborting.")
        break
    src = random.choice(images)
    img = cv2.imread(src)
    if img is None:
        continue

    label_path = find_label_for_image(src, labels_dir)
    if label_path is None:
        continue

    boxes_raw = read_label_file(label_path)
    if not boxes_raw:
        continue

    bboxes = [coords for (cls, coords) in boxes_raw]
    class_labels = [cls for (cls, coords) in boxes_raw]

    try:
        augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
    except Exception:
        continue

    aug_bboxes = augmented.get("bboxes", [])
    aug_labels = augmented.get("class_labels", [])
    aug_image = augmented["image"]

    if not aug_bboxes:
        continue

    src_stem = os.path.splitext(os.path.basename(src))[0]

```

```

new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
ext = os.path.splitext(src)[1]
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue
clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    if x2 - x1 < 1 or y2 - y1 < 1:
        continue
    clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue

write_label_file(new_label_path, clipped_boxes)
generated += 1
if generated % 10 == 0 or generated == needed:
    print(f"Generated {generated}/{needed}")

print(f"Done: generated {generated} augmented images (attempts {attempts})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--labels_dir", type=str, default=r"D:\Dataset\train\Cheetah\Label",
help="Path to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

```

```

def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]
            try:
                coords = list(map(float, parts[1:5]))
            except Exception:
                continue
            boxes.append((cls, coords))
    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

```

```

def build_transform():
    return A.Compose(
        [
            A.OneOf([
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.1),
            ], p=0.5),
            A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.2, rotate_limit=20,
border_mode=cv2.BORDER_CONSTANT, p=0.7),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
            A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10,
p=0.3),
            A.ImageCompression(quality_lower=75, quality_upper=95, p=0.3),
        ],
def main(images_dir, labels_dir, target_count=400, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    # ensure output folder exists
    os.makedirs(images_dir, exist_ok=True)

```

```

while generated < needed and attempts < needed * max_tries:
    attempts += 1
    if not images:
        print("No source images available to augment. Aborting.")
        break
    src = random.choice(images)
    img = cv2.imread(src)
    if img is None:
        continue

    label_path = find_label_for_image(src, labels_dir)
    if label_path is None:
        continue

    boxes_raw = read_label_file(label_path)
    if not boxes_raw:
        continue

    bboxes = [coords for (cls, coords) in boxes_raw]
    class_labels = [cls for (cls, coords) in boxes_raw]

    try:
        augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
    except Exception:
        continue

    aug_bboxes = augmented.get("bboxes", [])
    aug_labels = augmented.get("class_labels", [])
    aug_image = augmented["image"]

    if not aug_bboxes:
        continue

    src_stem = os.path.splitext(os.path.basename(src))[0]
    new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
    ext = os.path.splitext(src)[1]
    new_image_path = os.path.join(images_dir, new_stem + ext)
    new_label_path = os.path.join(labels_dir, new_stem + ".txt")

    success = cv2.imwrite(new_image_path, aug_image)
    if not success:
        continue

    clipped_boxes = []

```



```

    for lbl, box in zip(aug_labels, aug_bboxes):
        x1, y1, x2, y2 = box
        x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
        x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
        y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
        y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
        if x2 - x1 < 1 or y2 - y1 < 1:
            continue
        clipped_boxes.append((lbl, [x1, y1, x2, y2]))
    if not clipped_boxes:
        try:
            os.remove(new_image_path)
        except Exception:
            pass
        continue

    write_label_file(new_label_path, clipped_boxes)
    generated += 1
    if generated % 10 == 0 or generated == needed:
        print(f"Generated {generated}/{needed}")

    print(f"Done: generated {generated} augmented images (attempts {attempts})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")

    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Crocodile\\Label",
help="Path to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:

```

```

        line = line.strip()
        if not line:
            continue
        parts = line.split()
        if len(parts) < 5:
            continue
        cls = parts[0]
        try:
            coords = list(map(float, parts[1:5]))
        except Exception:
            continue
        boxes.append((cls, coords))

    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def build_transform():
    return A.Compose(
        [
            A.OneOf([
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.1),
            ])
        ]
    )

```

```

        ], p=0.5),
        A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.2, rotate_limit=20,
border_mode=cv2.BORDER_CONSTANT, p=0.7),
        A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
        A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10,
p=0.3),
        A.ImageCompression(quality_lower=75, quality_upper=95, p=0.3),
    ],
def main(images_dir, labels_dir, target_count=400, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    # ensure output folder exists
    os.makedirs(images_dir, exist_ok=True)

    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        if not images:
            print("No source images available to augment. Aborting.")
            break
        src = random.choice(images)
        img = cv2.imread(src)
        if img is None:
            continue

```

```

label_path = find_label_for_image(src, labels_dir)
if label_path is None:
    continue

boxes_raw = read_label_file(label_path)
if not boxes_raw:
    continue

bboxes = [coords for (cls, coords) in boxes_raw]
class_labels = [cls for (cls, coords) in boxes_raw]

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception:
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    continue

src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f'{src_stem}_aug_{uuid.uuid4().hex[:8]}'
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue

clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    if x2 - x1 < 1 or y2 - y1 < 1:
        continue

```

```

        clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue

write_label_file(new_label_path, clipped_boxes)
generated += 1
if generated % 10 == 0 or generated == needed:
    print(f"Generated {generated}/{needed}")

print(f"Done: generated {generated} augmented images (attempts {attempts})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Elephant\\Label",
help="Path to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)
def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]
            try:
                coords = list(map(float, parts[1:5]))

```

```

        except Exception:

            continue

        boxes.append((cls, coords))

def write_label_file(path, boxes):
    for cls, coords in boxes:
        coords_str = " ".join(f"{c:.6f}" for c in coords)
        f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def build_transform():
    return A.Compose(
        [
            A.OneOf([
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.1),
            ], p=0.5),
            A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.2, rotate_limit=20,
border_mode=cv2.BORDER_CONSTANT, p=0.7),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
            A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10,
p=0.3),
            A.ImageCompression(quality_lower=75, quality_upper=95, p=0.3),
        ],
    )

```

```

def main(images_dir, labels_dir, target_count=430, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f"Found {existing} images in {images_dir}")
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f"Need to generate {needed} augmented images")

    transform = build_transform()

    generated = 0
    attempts = 0
    # ensure output folder exists
    os.makedirs(images_dir, exist_ok=True)

    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        if not images:
            print("No source images available to augment. Aborting.")
            break
        src = random.choice(images)
        img = cv2.imread(src)
        if img is None:
            continue

        label_path = find_label_for_image(src, labels_dir)
        if label_path is None:
            continue

        boxes_raw = read_label_file(label_path)
        if not boxes_raw:
            continue

        bboxes = [coords for (cls, coords) in boxes_raw]
        class_labels = [cls for (cls, coords) in boxes_raw]

```

```

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception:
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    continue

src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue

clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    if x2 - x1 < 1 or y2 - y1 < 1:
        continue
    clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue

write_label_file(new_label_path, clipped_boxes)

```



```

generated += 1
if generated % 10 == 0 or generated == needed:
    print(f'Generated {generated}/{needed}')

print(f'Done: generated {generated} augmented images (attempts {attempts})')

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Fox\\Label", help="Path
to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]
            try:
                coords = list(map(float, parts[1:5]))
            except Exception:
                continue
            boxes.append((cls, coords))
    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:

```

```

        coords_str = " ".join(f'{c:.6f}' for c in coords)
        f.write(f'{cls} {coords_str}\n')

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return Non

def main(images_dir, labels_dir, target_count=450, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = [".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    os.makedirs(images_dir, exist_ok=True)

    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        if not images:
            print("No source images available to augment. Aborting.")
            break

```

```

src = random.choice(images)
img = cv2.imread(src)
if img is None:
    continue

label_path = find_label_for_image(src, labels_dir)
if label_path is None:
    continue

boxes_raw = read_label_file(label_path)
if not boxes_raw:
    continue

bboxes = [coords for (cls, coords) in boxes_raw]
class_labels = [cls for (cls, coords) in boxes_raw]

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception:
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    continue

src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue

clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))

```

```

        y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
        y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
        if x2 - x1 < 1 or y2 - y1 < 1:
            continue
        clipped_boxes.append((lbl, [x1, y1, x2, y2]))
    if not clipped_boxes:
        try:
            os.remove(new_image_path)
        except Exception:
            pass
        continue

    write_label_file(new_label_path, clipped_boxes)
    generated += 1
    if generated % 10 == 0 or generated == needed:
        print(f'Generated {generated}/{needed}')

    print(f'Done: generated {generated} augmented images (attempts {attempts})')

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Leopard\\Label",
help="Path to labels folder")
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]

```

```

        try:
            coords = list(map(float, parts[1:5]))
        except Exception:
            continue
        boxes.append((cls, coords))

    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def build_transform():
    return A.Compose(
        [
            A.OneOf([
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.1),
            ], p=0.5),
            A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.2, rotate_limit=20,
border_mode=cv2.BORDER_CONSTANT, p=0.7),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
            A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10, p=0.3),
            A.ImageCompression(quality_lower=75, quality_upper=95, p=0.3),
        ],
    )

```

```

def main(images_dir, labels_dir, target_count=500, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    os.makedirs(images_dir, exist_ok=True)

    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        if not images:
            print("No source images available to augment. Aborting.")
            break
        src = random.choice(images)
        img = cv2.imread(src)
        if img is None:
            continue

        label_path = find_label_for_image(src, labels_dir)
        if label_path is None:
            continue

        boxes_raw = read_label_file(label_path)
        if not boxes_raw:
            continue

        bboxes = [coords for (cls, coords) in boxes_raw]
        class_labels = [cls for (cls, coords) in boxes_raw]

```

```

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception:
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    continue

src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue
clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    if x2 - x1 < 1 or y2 - y1 < 1:
        continue
    clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue

write_label_file(new_label_path, clipped_boxes)
generated += 1
if generated % 10 == 0 or generated == needed:

```

```

print(f"Generated {generated}/{needed}")

print(f"Done: generated {generated} augmented images (attempts {attempts})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--images_dir", type=str, default=r"D:\\Dataset\\train\\Lion", help="Path to
images folder")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Lion\\Label",
help="Path to labels folder")
    parser.add_argument("--target", type=int, default=500, help="Desired total image count")
    parser.add_argument("--seed", type=int, default=42)
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 5:
                continue
            cls = parts[0]
            try:
                coords = list(map(float, parts[1:5]))
            except Exception:
                continue
            boxes.append((cls, coords))
    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:

```



```

    for cls, coords in boxes:
        coords_str = " ".join(f"{c:.6f}" for c in coords)
        f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None

def main(images_dir, labels_dir, target_count=450, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f'Found {existing} images in {images_dir}')
    if existing >= target_count:
        print("Already at or above target count. Nothing to do.")
        return

    needed = target_count - existing
    print(f'Need to generate {needed} augmented images')

    transform = build_transform()

    generated = 0
    attempts = 0
    os.makedirs(images_dir, exist_ok=True)

    while generated < needed and attempts < needed * max_tries:
        attempts += 1
        if not images:

```

```

    print("No source images available to augment. Aborting.")
    break
src = random.choice(images)
img = cv2.imread(src)
if img is None:
    continue

label_path = find_label_for_image(src, labels_dir)
if label_path is None:
    continue

boxes_raw = read_label_file(label_path)
if not boxes_raw:
    continue

bboxes = [coords for (cls, coords) in boxes_raw]
class_labels = [cls for (cls, coords) in boxes_raw]

try:
    augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
except Exception:
    continue

aug_bboxes = augmented.get("bboxes", [])
aug_labels = augmented.get("class_labels", [])
aug_image = augmented["image"]

if not aug_bboxes:
    continue

src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f"{src_stem}_aug_{uuid.uuid4().hex[:8]}"
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")

success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue

clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box

```

```

x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
if x2 - x1 < 1 or y2 - y1 < 1:
    continue
    clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue

write_label_file(new_label_path, clipped_boxes)
generated += 1
if generated % 10 == 0 or generated == needed:
    print(f"Generated {generated}/{needed}")

print(f"Done: generated {generated} augmented images (attempts {attempts})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label
files (pascal_voc style)")
    parser.add_argument("--images_dir", type=str, default=r"D:\\Dataset\\train\\Pig", help="Path to
images folder")
    parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Pig\\Label", help="Path
to labels folder")
    parser.add_argument("--target", type=int, default=450, help="Desired total image count")
    parser.add_argument("--seed", type=int, default=42)
    args = parser.parse_args()

    main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)
def read_label_file(path):
    boxes = []
    if not os.path.exists(path):
        return boxes
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()

```

```

        if not line:
            continue
        parts = line.split()
        if len(parts) < 5:
            continue
        cls = parts[0]
        try:
            coords = list(map(float, parts[1:5]))
        except Exception:
            continue
        boxes.append((cls, coords))

    return boxes

def write_label_file(path, boxes):
    with open(path, "w", encoding="utf-8") as f:
        for cls, coords in boxes:
            coords_str = " ".join(f"{c:.6f}" for c in coords)
            f.write(f"{cls} {coords_str}\n")

def find_label_for_image(image_path, labels_dir):
    stem = os.path.splitext(os.path.basename(image_path))[0]
    candidates = [
        os.path.join(labels_dir, stem + ext)
        for ext in [".txt", ".TXT"]
    ]
    for c in candidates:
        if os.path.exists(c):
            return c
    for p in glob.glob(os.path.join(labels_dir, stem + "*.txt")):
        return p
    return None+

def main(images_dir, labels_dir, target_count=500, seed=42, max_tries=10):
    random.seed(seed)

    img_patterns = ["*.jpg", "*.jpeg", "*.png", "*.bmp", "*.tif", "*.tiff"]
    images = []
    for p in img_patterns:
        images.extend(glob.glob(os.path.join(images_dir, p)))
    images = sorted(images)
    existing = len(images)
    print(f"Found {existing} images in {images_dir}")

```

```

if existing >= target_count:
    print("Already at or above target count. Nothing to do.")
    return

needed = target_count - existing
print(f'Need to generate {needed} augmented images')

transform = build_transform()

generated = 0
attempts = 0
os.makedirs(images_dir, exist_ok=True)

while generated < needed and attempts < needed * max_tries:
    attempts += 1
    if not images:
        print("No source images available to augment. Aborting.")
        break
    src = random.choice(images)
    img = cv2.imread(src)
    if img is None:
        continue

    label_path = find_label_for_image(src, labels_dir)
    if label_path is None:
        continue

    boxes_raw = read_label_file(label_path)
    if not boxes_raw:
        continue

    bboxes = [coords for (cls, coords) in boxes_raw]
    class_labels = [cls for (cls, coords) in boxes_raw]

    try:
        augmented = transform(image=img, bboxes=bboxes, class_labels=class_labels)
    except Exception:
        continue

    aug_bboxes = augmented.get("bboxes", [])
    aug_labels = augmented.get("class_labels", [])
    aug_image = augmented["image"]

    if not aug_bboxes:

```

```
    continue
```

```
src_stem = os.path.splitext(os.path.basename(src))[0]
new_stem = f'{src_stem}_aug_{uuid.uuid4().hex[:8]}'
ext = os.path.splitext(src)[1]
new_image_path = os.path.join(images_dir, new_stem + ext)
new_label_path = os.path.join(labels_dir, new_stem + ".txt")
```

```
success = cv2.imwrite(new_image_path, aug_image)
if not success:
    continue
```

```
clipped_boxes = []
for lbl, box in zip(aug_labels, aug_bboxes):
    x1, y1, x2, y2 = box
    x1 = max(0.0, min(x1, aug_image.shape[1] - 1))
    x2 = max(0.0, min(x2, aug_image.shape[1] - 1))
    y1 = max(0.0, min(y1, aug_image.shape[0] - 1))
    y2 = max(0.0, min(y2, aug_image.shape[0] - 1))
    if x2 - x1 < 1 or y2 - y1 < 1:
        continue
    clipped_boxes.append((lbl, [x1, y1, x2, y2]))
if not clipped_boxes:
    try:
        os.remove(new_image_path)
    except Exception:
        pass
    continue
```

```
write_label_file(new_label_path, clipped_boxes)
generated += 1
if generated % 10 == 0 or generated == needed:
    print(f'Generated {generated}/{needed}')
```

```
print(f'Done: generated {generated} augmented images (attempts {attempts})')
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate augmented images and matching label files (pascal_voc style)")
    parser.add_argument("--images_dir", type=str, default=r"D:\Dataset\train\Tiger", help="Path to images folder")
```

```

parser.add_argument("--labels_dir", type=str, default=r"D:\\Dataset\\train\\Tiger\\Label",
help="Path to labels folder")
parser.add_argument("--target", type=int, default=500, help="Desired total image count")
parser.add_argument("--seed", type=int, default=42)
args = parser.parse_args()

main(args.images_dir, args.labels_dir, target_count=args.target, seed=args.seed)

```

YOLO Format Conversion

```

"""

```

Convert Pascal VOC format labels to YOLO11 format.

Pascal VOC: class_name x1 y1 x2 y2 (pixel coordinates)

YOLO11: class_id x_center y_center width height (normalized 0-1)

```

"""

```

```

import os
import cv2
from pathlib import Path

```

Map class names to class IDs

```

CLASS_MAPPING = {

```

```

    'Bear': 0,
    'Cheetah': 1,
    'Crocodile': 2,
    'Eagle': 3,
    'Elephant': 4,
    'Fox': 5,
    'Leopard': 6,
    'Lion': 7,
    'Monkey': 8,
    'Pig': 9,
    'Scorpion': 10,
    'Snake': 11,
    'Tiger': 12,

```

```

}

```

```

def pascal_to_yolo(x1, y1, x2, y2, img_width, img_height):

```

```

    """

```

Convert Pascal VOC coordinates to YOLO format.

Args:

x1, y1, x2, y2: Bounding box coordinates in pixels (top-left and bottom-right)

img_width, img_height: Image dimensions in pixels

Returns:

x_center, y_center, width, height: Normalized YOLO format coordinates
"""

```
# Convert from pixel to normalized coordinates
x_center = ((x1 + x2) / 2) / img_width
y_center = ((y1 + y2) / 2) / img_height
width = (x2 - x1) / img_width
height = (y2 - y1) / img_height
# Clip values to ensure they're within [0, 1]
x_center = max(0, min(1, x_center))
y_center = max(0, min(1, y_center))
width = max(0, min(1, width))
height = max(0, min(1, height))
return x_center, y_center, width, height
```

```
def convert_labels(dataset_path, split='train'):
    dataset_path = Path(dataset_path)
    split_path = dataset_path / split
    if not split_path.exists():
        print(f"Error: {split_path} does not exist")
        return
    total_converted = 0
    errors = 0
    # Process each animal class
    for class_name in CLASS_MAPPING.keys():
        class_path = split_path / class_name
        label_dir = class_path / 'Label'

        if not label_dir.exists():
            print(f"Warning: {label_dir} does not exist, skipping {class_name}")
            continue

        class_id = CLASS_MAPPING[class_name]
        label_files = sorted(label_dir.glob('*.*txt'))
```



```

print(f"\nProcessing {class_name} ({len(label_files)} labels)...")

for label_file in label_files:
    try:
        # Read the label file
        with open(label_file, 'r') as f:
            lines = f.readlines()

        # Find corresponding image file
        # Try different image extensions
        image_name = label_file.stem
        image_path = None
        for ext in ['.jpg', '.jpeg', '.png', '.JPG', '.JPEG', '.PNG']:
            potential_path = class_path / f'{image_name}{ext}'
            if potential_path.exists():
                image_path = potential_path
                break

        if image_path is None:
            print(f" Warning: No image found for {label_file.name}")
            errors += 1
            continue

        # Read image to get dimensions
        img = cv2.imread(str(image_path))
        if img is None:
            print(f" Warning: Could not read image {image_path}")
            errors += 1
            continue

        # Convert each bounding box in the label file
        yolo_lines = []
        for line in lines:
            parts = line.strip().split()
            if len(parts) < 5:
                continue

            label_class = parts[0]
            try:
                x1, y1, x2, y2 = map(float, parts[1:5])
            except ValueError:
                print(f" Warning: Invalid coordinates in {label_file.name}")
                errors += 1

```

```

        continue

    # Verify class name matches
    if label_class != class_name:
        print(f" Warning: Class mismatch in {label_file.name}: {label_class} vs
{class_name}")

    # Convert to YOLO format
    x_center, y_center, width, height = pascal_to_yolo(
        x1, y1, x2, y2, img_width, img_height
    )

    # Format: class_id x_center y_center width height
    yolo_line = f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}\n"
    yolo_lines.append(yolo_line)

    # Write converted labels back to file
    with open(label_file, 'w') as f:
        f.writelines(yolo_lines)

    total_converted += 1

except Exception as e:
    print(f" Error processing {label_file.name}: {str(e)}")
    errors += 1

print(f"\n{'='*60}")
print(f"Conversion complete for '{split}' split!")
print(f"Total labels converted: {total_converted}")
print(f"Errors: {errors}")
print(f"{'='*60}")

if __name__ == '__main__':
    dataset_path = r'd:\Dataset'

    # Convert both train and test splits
    for split in ['train', 'test']:
        convert_labels(dataset_path, split)

    print("\nAll conversions completed!")

```

Training.ipynb

```
#Install Ultralytics to train Yolo model
!pip install -U ultralytics
```

Collecting ultralytics

Downloading ultralytics-8.4.21-py3-none-any.whl.metadata (39 kB)

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)

Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)

Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)

Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)

Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)

Collecting ultralytics-thop>=2.0.18 (from ultralytics)

Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)

Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2025.3.0)

Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.9.4)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in

```

/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.7.3.90)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-
packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages
(from jinja2->torch>=1.8.0->ultralytics) (3.0.3)
Downloading ultralytics-8.4.21-py3-none-any.whl (1.2 MB)

```

```

——— 1.2/1.2 MB 31.1 MB/s eta 0:00:00

```

```

Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)

```

```

Installing collected packages: ultralytics-thop, ultralytics

```

```

Successfully installed ultralytics-8.4.21 ultralytics-thop-2.0.18

```

```

from ultralytics import YOLO

```

```

# Load pretrained YOLOv11
model = YOLO("yolo11n.pt")

```

```

# Train
model.train(
    data="/content/drive/MyDrive/Dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,      # T4 GPU safe

```

```

device=0,      # GPU
project="runs/train",
name="wild_animal_yolov11"
)

```

Ultralytics 8.4.12 🚀 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False,
auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None,
close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip,
cos_lr=False, cutmix=0.0, data=/content/drive/MyDrive/Dataset/data.yaml, degrees=0.0,
deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None,
end2end=None, epochs=50, erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0,
format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4,
imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolo11n.pt, momentum=0.937,
mosaic=1.0, multi_scale=0.0, name=wild_animal_yolov112, nbs=64, nms=False, opset=None,
optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True,
pose=12.0, pretrained=True, profile=False, project=runs/train, rect=False, resume=False,
retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False

Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% —
————— 755.1KB 22.6MB/s 0.0s

Transferred 448/499 items from pretrained weights

Freezing layer 'model.23.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to
'yolo26n.pt': 100% ————— 5.3MB 97.2MB/s 0.1s

AMP: checks passed ✅

train: Fast image access ✅ (ping: 1.1±1.4 ms, read: 0.5±0.2 MB/s, size: 237.3 KB)

train: Scanning /content/drive/MyDrive/Dataset/train/labels... 5559 images, 0 backgrounds, 0
corrupt: 100% ————— 5559/5559 2.1it/s 43:32

train: New cache created: /content/drive/MyDrive/Dataset/train/labels.cache

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

val: Fast image access  (ping: 1.1±0.7 ms, read: 1.0±0.5 MB/s, size: 626.0 KB)

val: Scanning /content/drive/MyDrive/Dataset/val/labels... 1255 images, 0 backgrounds, 0 corrupt: 100%

 1255/1255 2.2it/s 9:33

val: New cache created: /content/drive/MyDrive/Dataset/val/labels.cache

Plotting labels to /content/runs/detect/runs/train/wild_animal_yolov112/labels.jpg...

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...

optimizer: AdamW(lr=0.000588, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)

Image sizes 640 train, 640 val

Using 2 dataloader workers

Logging results to /content/runs/detect/runs/train/wild_animal_yolov112

Starting training for 50 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/50	2.21G	0.8081	3.045	1.351	18	640: 100%

348/348 1.0it/s 5:38

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
40/40 2.3s/it 1:32						
all	1255	1375	0.51	0.594	0.556	0.42

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/50	3.26G	0.8523	1.998	1.35	17	640: 100%

348/348 2.2it/s 2:39

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
40/40 1.8it/s 21.7s						
all	1255	1375	0.955	0.968	0.991	0.94

Closing dataloader mosaic

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
-------	---------	----------	----------	----------	-----------	------

```

41/50  3.84G  0.4224  0.3889  1.052    7   640: 100% —————
— 348/348 2.2it/s 2:37
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 1.9it/s 21.0s
      all   1255   1375   0.974   0.938   0.982   0.928
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
42/50  3.85G  0.3973  0.3447  1.022    8   640: 100% —————
— 348/348 2.3it/s 2:31
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 2.1it/s 19.3s
      all   1255   1375   0.983   0.945   0.987   0.935
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
43/50  3.87G  0.3905  0.3253  1.014    7   640: 100% —————
— 348/348 2.3it/s 2:33
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 2.1it/s 19.2s
      all   1255   1375   0.986   0.969   0.991   0.946
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
44/50  3.88G  0.3808  0.3154  1.009    8   640: 100% —————
— 348/348 2.2it/s 2:36
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 2.0it/s 20.0s
      all   1255   1375   0.992   0.972   0.992   0.947
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
45/50  3.89G  0.3743  0.3083  1.005    7   640: 100% —————
— 348/348 2.3it/s 2:34
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 2.0it/s 20.5s
      all   1255   1375   0.977   0.967   0.992   0.948
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
46/50  3.91G  0.3647  0.299   0.9997    7   640: 100% —————
— 348/348 2.3it/s 2:33
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 1.8it/s 22.4s
      all   1255   1375   0.974   0.971   0.99   0.948
Epoch GPU_mem box_loss cls_loss dfl_loss Instances  Size
47/50  3.92G  0.3518  0.285   0.9838    7   640: 100% —————
— 348/348 2.3it/s 2:35
      Class  Images Instances  Box(P    R   mAP50 mAP50-95): 100% —————
      ——— 40/40 1.9it/s 20.6s
      all   1255   1375   0.986   0.96   0.988   0.951

```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
48/50	3.94G	0.3468	0.285	0.9754	7	640: 100%

— 348/348 2.3it/s 2:29

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	1255	1375	0.97	0.966	0.989	0.95

40/40 2.0it/s 19.7s

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
49/50	3.95G	0.3404	0.2734	0.9774	7	640: 100%

— 348/348 2.3it/s 2:31

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	1255	1375	0.964	0.981	0.989	0.955

40/40 2.0it/s 19.6s

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
50/50	3.97G	0.3355	0.2639	0.9708	7	640: 100%

— 348/348 2.3it/s 2:30

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	1255	1375	0.984	0.969	0.993	0.958

40/40 1.9it/s 21.1s

50 epochs completed in 2.543 hours.

Optimizer stripped from /content/runs/detect/runs/train/wild_animal_yolov112/weights/last.pt, 5.5MB

Optimizer stripped from /content/runs/detect/runs/train/wild_animal_yolov112/weights/best.pt, 5.5MB

Validating /content/runs/detect/runs/train/wild_animal_yolov112/weights/best.pt...

Ultralytics 8.4.12 🚀 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)

YOLO11n summary (fused): 101 layers, 2,584,687 parameters, 0 gradients, 6.3 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	1255	1375	0.984	0.969	0.993	0.958
Bear	39	42	0.996	1	0.995	0.968
Cheetah	6	6	0.846	1	0.995	0.977
Crocodile	76	81	1	0.988	0.995	0.925
Eagle	178	190	1	0.98	0.995	0.976
Elephant	33	39	1	0.973	0.994	0.963
Fox	69	72	0.979	1	0.995	0.971
Leopard	55	59	1	0.793	0.995	0.976
Lion	99	103	0.997	1	0.995	0.975
Monkey	321	365	1	0.953	0.994	0.954
Pig	96	128	0.977	0.992	0.994	0.944

40/40 1.9it/s 21.6s

Scorpion	44	44	0.998	1	0.995	0.946
Snake	213	219	1	0.957	0.989	0.945
Tiger	26	27	0.996	0.963	0.975	0.937

Speed: 0.2ms preprocess, 2.6ms inference, 0.0ms loss, 2.6ms postprocess per image

Results saved to **/content/runs/detect/runs/train/wild_animal_yolov112**

ultralytics.utils.metrics.DetMetrics object with attributes:

0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26927, 0.27027, 0.27127,
0.27227, 0.27327, 0.27427, 0.27528, 0.27628, 0.27728, 0.27828, 0.27928,
0.28028, 0.28128, 0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,
0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29329, 0.29429, 0.2953,
0.2963, 0.2973, 0.2983, 0.2993, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043,
0.30531, 0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,
0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31732, 0.31832, 0.31932,
0.32032, 0.32132, 0.32232, 0.32332, 0.32432, 0.32533, 0.32633, 0.32733,
0.32833, 0.32933, 0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,
0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34134, 0.34234, 0.34334,
0.34434, 0.34535, 0.34635, 0.34735, 0.34835, 0.34935, 0.35035, 0.35135,
0.35235, 0.35335, 0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,
0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36537, 0.36637, 0.36737,
0.36837, 0.36937, 0.37037, 0.37137, 0.37237, 0.37337, 0.37437, 0.37538,
0.37638, 0.37738, 0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,
0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38939, 0.39039, 0.39139,
0.39239, 0.39339, 0.39439, 0.3954, 0.3964, 0.3974, 0.3984, 0.3994, 0.4004,
0.4014, 0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77377, 0.77477, 0.77578,
0.77678, 0.77778, 0.77878, 0.77978, 0.78078, 0.78178, 0.78278, 0.78378,
0.78478, 0.78579, 0.78679, 0.78779, 0.78879, 0.78979, 0.79079, 0.79179,
0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7978, 0.7988, 0.7998,
0.8008, 0.8018, 0.8028, 0.8038, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881,
0.80981, 0.81081, 0.81181, 0.81281, 0.81381, 0.81481, 0.81582,
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82182, 0.82282, 0.82382,
0.82482, 0.82583, 0.82683, 0.82783, 0.82883, 0.82983, 0.83083, 0.83183,
0.83283, 0.83383, 0.83483, 0.83584, 0.83684, 0.83784, 0.83884, 0.83984,
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84585, 0.84685, 0.84785,
0.84885, 0.84985, 0.85085, 0.85185, 0.85285, 0.85385, 0.85485, 0.85586,
0.85686, 0.85786, 0.85886, 0.85986, 0.86086, 0.86186, 0.86286, 0.86386,
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86987, 0.87087, 0.87187,
0.87287, 0.87387, 0.87487, 0.87588, 0.87688, 0.87788, 0.87888, 0.87988,
0.88088, 0.88188, 0.88288, 0.88388, 0.88488, 0.88589, 0.88689, 0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89389, 0.89489, 0.8959,
0.8969, 0.8979, 0.8989, 0.8999, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049,
0.90591, 0.90691, 0.90791, 0.90891, 0.90991, 0.91091, 0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91792, 0.91892, 0.91992,

0.92092, 0.92192, 0.92292, 0.92392, 0.92492, 0.92593, 0.92693, 0.92793,
 0.92893, 0.92993, 0.93093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
 0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94194, 0.94294, 0.94394,
 0.94494, 0.94595, 0.94695, 0.94795, 0.94895, 0.94995, 0.95095, 0.95195,
 0.95295, 0.95395, 0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
 0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96597, 0.96697, 0.96797,
 0.96897, 0.96997, 0.97097, 0.97197, 0.97297, 0.97397, 0.97497, 0.97598,
 0.97698, 0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
 0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98999, 0.99099, 0.99199,
 0.99299, 0.99399, 0.99499, 0.996, 0.997, 0.998, 0.999, 1]), array([[1,
 1, 1, ..., 1, 1, 0],
 [1, 1, 1, ..., 1, 1, 0],
 [1, 1, 1, ..., 0.9759, 0.9759, 0],
 ...,
 [1, 1, 1, ..., 1, 1, 0],
 [1, 1, 1, ..., 0.15746, 0.078731, 0],
 [1, 1, 1, ..., 0.32927, 0.32927, 0]]),
 0.42442, 0.42543, 0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,
 0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43744, 0.43844, 0.43944,
 0.44044, 0.44144, 0.44244, 0.44344, 0.44444, 0.44545, 0.44645, 0.44745,
 0.44845, 0.44945, 0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,
 0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.46146, 0.46246, 0.46346,
 0.46446, 0.46547, 0.46647, 0.46747, 0.46847, 0.46947, 0.47047, 0.47147,
 0.47247, 0.47347, 0.47447, 0.47548, 0.47648, 0.47748, 0.47848, 0.47948,
 0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94194, 0.94294, 0.94394,
 0.94494, 0.94595, 0.94695, 0.94795, 0.94895, 0.94995, 0.95095, 0.95195,
 0.95295, 0.95395, 0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
 0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96597, 0.96697, 0.96797,
 0.96897, 0.96997, 0.97097, 0.97197, 0.97297, 0.97397, 0.97497, 0.97598,
 0.97698, 0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
 0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98999, 0.99099, 0.99199,
 0.99299, 0.99399, 0.99499, 0.996, 0.997, 0.998, 0.999, 1]), array([[
 0.43523, 0.43523, 0.64841, ..., 0, 0, 0],
 [0.10345, 0.10345, 0.12526, ..., 0, 0, 0],
 [0.49091, 0.49091, 0.62768, ..., 0, 0, 0],
 ...,
 [0.73333, 0.73333, 0.85153, ..., 0, 0, 0],
 [0.52785, 0.52785, 0.66452, ..., 0, 0, 0],
 [0.39706, 0.39706, 0.55078, ..., 0, 0, 0]]),
 0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36537, 0.36637, 0.36737,
 0.36837, 0.36937, 0.37037, 0.37137, 0.37237, 0.37337, 0.37437, 0.37538,
 0.37638, 0.37738, 0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,
 0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38939, 0.39039, 0.39139,
 0.39239, 0.39339, 0.39439, 0.3954, 0.3964, 0.3974, 0.3984, 0.3994, 0.4004,

0.4014, 0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,
 0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.41341, 0.41441, 0.41542,
 0.41642, 0.41742, 0.41842, 0.41942, 0.42042, 0.42142, 0.42242, 0.42342,
 0.42442, 0.42543, 0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,
 0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43744, 0.43844, 0.43944,
 0.44044, 0.44144, 0.44244, 0.44344, 0.44444, 0.44545, 0.44645, 0.44745,
 0.44845, 0.44945, 0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,
 0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.46146, 0.46246, 0.46346,
 0.46446, 0.46547, 0.46647, 0.46747, 0.46847, 0.46947, 0.47047, 0.47147,
 0.47247, 0.47347, 0.47447, 0.47548, 0.47648, 0.47748, 0.47848, 0.47948,
 0.48048, 0.48148, 0.48248, 0.48348, 0.48448, 0.48549, 0.48649, 0.48749,
 0.48849, 0.48949, 0.49049, 0.49149, 0.49249, 0.49349, 0.49449, 0.4955,
 0.4965, 0.4975, 0.4985, 0.4995, 0.5005, 0.5015, 0.5025, 0.5035,
 0.5045, 0.50551, 0.50651, 0.50751, 0.50851, 0.50951, 0.51051, 0.51151,
 0.51251, 0.51351, 0.51451, 0.51552, 0.51652, 0.51752, 0.51852, 0.51952,
 0.52052, 0.52152, 0.52252, 0.52352, 0.52452, 0.52553, 0.52653, 0.52753,
 0.52853, 0.52953, 0.53053, 0.53153, 0.53253, 0.53353, 0.53453, 0.53554,
 0.53654, 0.53754, 0.53854, 0.53954, 0.54054, 0.54154, 0.54254, 0.54354,
 0.54454, 0.54555, 0.54655, 0.54755, 0.54855, 0.54955, 0.55055, 0.55155,
 0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91792, 0.91892, 0.91992,
 0.92092, 0.92192, 0.92292, 0.92392, 0.92492, 0.92593, 0.92693, 0.92793,
 0.92893, 0.92993, 0.93093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
 0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94194, 0.94294, 0.94394,
 0.94494, 0.94595, 0.94695, 0.94795, 0.94895, 0.94995, 0.95095, 0.95195,
 0.95295, 0.95395, 0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
 0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96597, 0.96697, 0.96797,
 0.96897, 0.96997, 0.97097, 0.97197, 0.97297, 0.97397, 0.97497, 0.97598,
 0.97698, 0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
 0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98999, 0.99099, 0.99199,
 0.99299, 0.99399, 0.99499, 0.996, 0.997, 0.998, 0.999, 1]), array([[
 0.27815, 0.27815, 0.47973, ..., 1, 1, 1],
 [0.054545, 0.054545, 0.066817, ..., 1, 1, 1],
 [0.3253, 0.3253, 0.45739, ..., 1, 1, 1],
 ...,
 [0.57895, 0.57895, 0.74145, ..., 1, 1, 1],
 [0.35914, 0.35914, 0.50105, ..., 1, 1, 1],
 [0.24771, 0.24771, 0.38569, ..., 1, 1, 1
 0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1011, 0.1021, 0.1031,
 0.1041, 0.10511, 0.10611, 0.10711, 0.10811, 0.10911, 0.11011, 0.11111,
 0.11211, 0.11311, 0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,
 0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77377, 0.77477, 0.77578,
 0.77678, 0.77778, 0.77878, 0.77978, 0.78078, 0.78178, 0.78278, 0.78378,
 0.78478, 0.78579, 0.78679, 0.78779, 0.78879, 0.78979, 0.79079, 0.79179,
 0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7978, 0.7988, 0.7998,

```

0.8008, 0.8018, 0.8028, 0.8038, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881,
0.80981, 0.81081, 0.81181, 0.81281, 0.81381, 0.81481, 0.81582,
    0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82182, 0.82282, 0.82382,
0.82482, 0.82583, 0.82683, 0.82783, 0.82883, 0.82983, 0.83083, 0.83183,
0.83283, 0.83383, 0.83483, 0.83584, 0.83684, 0.83784, 0.83884, 0.83984,
    0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84585, 0.84685, 0.84785,
0.84885, 0.84985, 0.85085, 0.85185, 0.85285, 0.85385, 0.85485, 0.85586,
0.85686, 0.85786, 0.85886, 0.85986, 0.86086, 0.86186, 0.86286, 0.86386,
    0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86987, 0.87087, 0.87187,
0.87287, 0.87387, 0.87487, 0.87588, 0.87688, 0.87788, 0.87888, 0.87988,
0.88088, 0.88188, 0.88288, 0.88388, 0.88488, 0.88589, 0.88689, 0.88789,
    0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89389, 0.89489, 0.8959,
0.8969, 0.8979, 0.8989, 0.8999, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049,
0.90591, 0.90691, 0.90791, 0.90891, 0.90991, 0.91091, 0.91191
    0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96597, 0.96697, 0.96797,
0.96897, 0.96997, 0.97097, 0.97197, 0.97297, 0.97397, 0.97497, 0.97598,
0.97698, 0.97798, 0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
    0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98999, 0.99099, 0.99199,
0.99299, 0.99399, 0.99499, 0.996, 0.997, 0.998, 0.999, 1]), array([[ 1,
1, 1, ..., 0, 0, 0],
[ 1, 1, 1, ..., 0, 0, 0],
[ 1, 1, 1, ..., 0, 0, 0],
...,
[ 1, 1, 1, ..., 0, 0, 0],
[ 0.99543, 0.99543, 0.9863, ..., 0, 0, 0],
[ 1, 1, 0.96296, ..., 0, 0, 0]]), 'Confidence', 'Recall'])
fitness: np.float64(0.9581832200667139)
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.96847, 0.97654, 0.92533, 0.97557, 0.96284, 0.97073, 0.97592,
0.97543, 0.95407, 0.94392, 0.94552, 0.94534, 0.9367])
names: {0: 'Bear', 1: 'Cheetah', 2: 'Crocodile', 3: 'Eagle', 4: 'Elephant', 5: 'Fox', 6: 'Leopard', 7: 'Lion',
8: 'Monkey', 9: 'Pig', 10: 'Scorpion', 11: 'Snake', 12: 'Tiger'}
nt_per_class: array([ 42, 6, 81, 190, 39, 72, 59, 103, 365, 128, 44, 219, 27])
nt_per_image: array([ 39, 6, 76, 178, 33, 69, 55, 99, 321, 96, 44, 213, 26])
results_dict: {'metrics/precision(B)': 0.9836095672723796, 'metrics/recall(B)': 0.9690929366926437,
'metrics/mAP50(B)': 0.99280520830342, 'metrics/mAP50-95(B)': 0.9581832200667139, 'fitness':
0.9581832200667139}
save_dir: PosixPath('/content/runs/detect/runs/train/wild_animal_yolov112')
speed: {'preprocess': 0.20965422948382229, 'inference': 2.597736208770084, 'loss':
0.0004989721155906305, 'postprocess': 2.552240891633713}

```

LOADING TRAINED MODEL

```

# Load trained model
!pip install -U ultralytics

```

```
from ultralytics import YOLO
import numpy as np
```

```
model =
YOLO("/content/drive/MyDrive/Dataset/runs/detect/runs/train/wild_animal_yolov112/weights/best.p
t")
```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.4.14)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (12.8.90)

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help
<https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

Testing

```
results = model.val(
    data="/content/drive/MyDrive/Dataset/data.yaml",
    split="test",
    imgsz=640,
    conf=0.25,
    iou=0.5,
    device="cpu" # or "0" if GPU available
)
metrics = results.box
precision = metrics.p.mean()
recall = metrics.r.mean()
map50 = metrics.map50
map5095 = metrics.map
f1 = 2 * (precision * recall) / (precision + recall + 1e-6)
print("===== TEST DATASET EVALUATION =====")
print(f"Precision (P):    {precision:.4f}")
print(f"Recall (R):        {recall:.4f}")
print(f"F1-score:           {f1:.4f}")
print(f"mAP@50:              {map50:.4f}")
```

```

results = model.val(
    data="/content/drive/MyDrive/Dataset/data.yaml",
    split="test",
    imgsz=640,
    conf=0.25,
    iou=0.5,
    device="cpu" # or "0" if GPU available
)
metrics = results.box
precision = metrics.p.mean()
recall = metrics.r.mean()
map50 = metrics.map50
map5095 = metrics.map
f1 = 2 * (precision * recall) / (precision + recall + 1e-6)
Ultralytics 8.4.14 Python-3.12.12 torch-2.9.0+cpu CPU (Intel Xeon CPU @ 2.20GHz)
val: Scanning /content/drive/MyDrive/Dataset/test/labels... 1255 images, 0 backgrounds, 0 corrupt:
100% ————— 1255/1255 55.0it/s 22.8s

```

val: New cache created: /content/drive/MyDrive/Dataset/test/labels.cache

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
————— 79/79 3.8s/it 4:57						
all	1255	1375	0.984	0.969	0.993	0.966
Bear	39	42	0.994	1	0.995	0.973
Cheetah	6	6	0.836	1	0.995	0.978
Crocodile	76	81	1	0.988	0.993	0.938
Eagle	178	190	1	0.981	0.995	0.979
Elephant	33	39	1	0.975	0.995	0.974
Fox	69	72	0.992	1	0.995	0.972
Leopard	55	59	1	0.789	0.995	0.978
Lion	99	103	0.996	1	0.995	0.979
Monkey	321	365	1	0.954	0.994	0.963
Pig	96	128	0.977	0.992	0.994	0.952
Scorpion	44	44	1	1	0.995	0.952
Snake	213	219	1	0.957	0.992	0.957
Tiger	26	27	1	0.963	0.981	0.961

===== TEST DATASET EVALUATION =====

```

Precision (P): 0.9843
Recall (R):    0.9691
F1-score:     0.9766
mAP@50:       0.9934

```

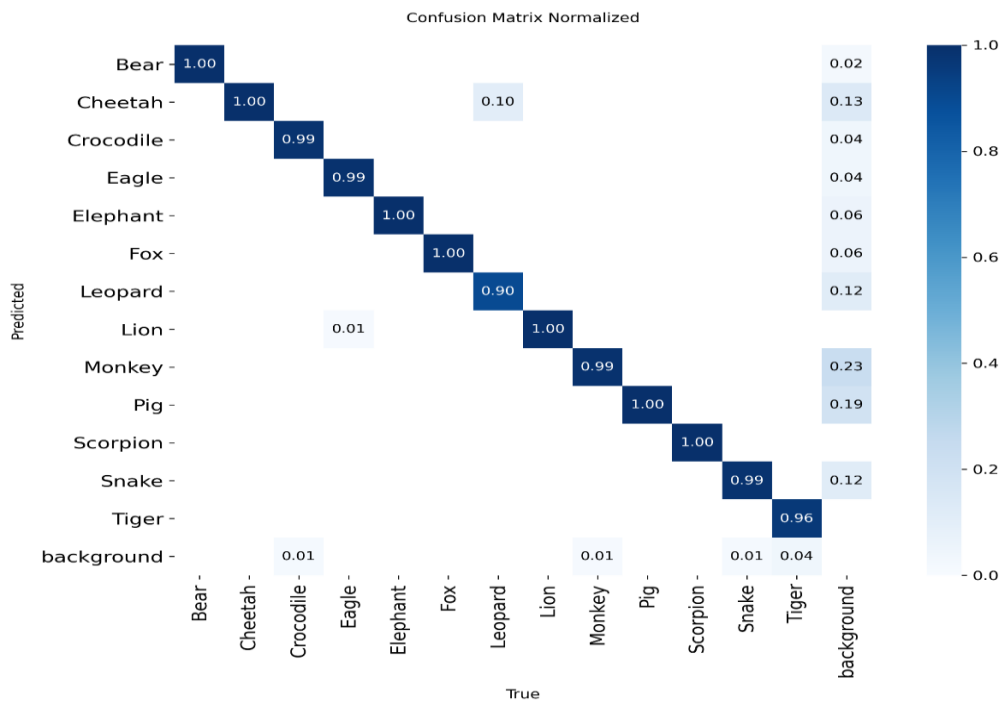


Fig 5.1 Confusion Matrix for Wild Animal Detection Model

CREATING USER INTERFACE

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Wild Animal Detection System</title>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;500;700&display=swap"
rel="stylesheet">
<style>
  body {
    font-family: 'Poppins', sans-serif;
    background: linear-gradient(135deg, #1f4037, #99f2c8);
    text-align: center;
    padding: 50px;
    color: white;
  }
  .container {
    background: rgba(0,0,0,0.6);
    border-radius: 15px;
```

```

width: 50%;
margin: auto;
    box-shadow: 0 10px 25px rgba(0,0,0,0.5);
}

h1 {
    margin-bottom: 20px;
}
input[type="file"] {
    padding: 10px;
    margin: 20px 0;
}
img {
    margin-top: 20px;
    width: 70%;
    border-radius: 10px;
    box-shadow: 0 5px 20px rgba(0,0,0,0.5);
}
.flash {
    margin-top: 15px;
    font-weight: bold;
    color: yellow;
}
.animal {
    font-size: 22px;
    margin-top: 15px;
    color: #ffcc00;
}
</style>
</head>
<body>
<div class="container">
    <h1> Livestock Farm Protection System</h1>
    <p>Upload a video to detect harmful wild animals</p>
    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="video" accept="video/*" required> <br>

```



```

        <button type="submit">Upload & Detect</button>
</form>
{% with messages = get_flashed_messages() %}
{% if messages %}
{% for message in messages %}
    <div class="flash">{{ message }}</div>
{% endfor %}
{% endif %}
{% endwith %}

{% if animal %}
    <div class="animal">
        Detected Animal : <b>{{ animal }}</b>
    </div>
{% endif %}

{% if image %}
    <h3>Detected Frame:</h3>
    
{% endif %}

</div>

</body>
</html>

```

App.py

```

import os
import cv2
import smtplib

from email.message import EmailMessage
from flask import Flask, render_template, request, redirect, url_for, flash
from ultralytics import YOLO as YoloModel

# =====
# Flask Setup
# =====
UPLOAD_FOLDER = "static/uploads"
DETECTED_FOLDER = "static/detected"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

```

```

os.makedirs(DETECTED_FOLDER, exist_ok=True)

# =====
# Load YOLO Model
# =====
model = YOLO("best.pt")

CONF_THRESHOLD = 0.5
CONSECUTIVE_FRAMES = 5

# =====
# Email Alert Function
# =====
def send_email_alert(image_path, animal_name):

    sender_email = "eamcet133@gmail.com"
    sender_password = "mmckjffoorsqjimpn"
    receiver_email = "patanarif74@gmail.com"

    msg = EmailMessage()
    msg["Subject"] = f"🚨 {animal_name} Detected in Farm!"
    msg["From"] = sender_email
    msg["To"] = receiver_email

    msg.set_content(
        f"""
Wild Animal Alert!

Detected Animal : {animal_name}

A harmful wild animal has been detected in the farm surveillance system.

Please check the attached frame immediately.
"""
    )

    with open(image_path, "rb") as f:
        file_data = f.read()
        file_name = os.path.basename(image_path)

    msg.add_attachment(file_data, maintype="image", subtype="jpeg", filename=file_name)

    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
        smtp.login(sender_email, sender_password)

```

```

smtp.send_message(msg)

# =====
# Main Route
# =====
@app.route("/", methods=["GET", "POST"])
def index():

    if request.method == "POST":

        video = request.files["video"]

        if video.filename == "":
            flash("Please upload a video file.")
            return redirect(request.url)

        video_path = os.path.join(UPLOAD_FOLDER, video.filename)
        video.save(video_path)

        cap = cv2.VideoCapture(video_path)

        detection_counter = 0
        detected_animal = None
        detected_image_path = None

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            results = model(frame, conf=CONF_THRESHOLD, verbose=False)
            detections = results[0].boxes

            if len(detections) > 0:

                detection_counter += 1

                class_id = int(detections.cls[0])
                detected_animal = model.names[class_id]

            else:
                detection_counter = 0

            if detection_counter >= CONSECUTIVE_FRAMES:

```

```

annotated_frame = results[0].plot()

detected_image_path = os.path.join(DETECTED_FOLDER, "detected.jpg")
cv2.imwrite(detected_image_path, annotated_frame)

send_email_alert(detected_image_path, detected_animal)

break

cap.release()

if detected_image_path:

    flash(f"🔔 {detected_animal} detected! Email alert sent.")

    return render_template(
        "index.html",
        image="detected/detected.jpg",
        animal=detected_animal
    )

else:

    flash("No wild animal detected.")
    return redirect(url_for("index"))

return render_template("index.html")

# =====
# Start the Application
# =====
if __name__ == "__main__":
    app.run(debug=False)

```

COMPARING BASE YOLOv11 vs. YOLOv11+TEMPORAL CONSISTENCE MECHANISM

Index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>YOLO Detection Comparison</title>

```

```
<style>
  body {
    font-family: Arial;
    background-color: #f4f6f9;
    text-align: center;
    padding: 40px;
  }

  h1 {
    color: #2c3e50;
  }

  .card {
    background: white;
    padding: 30px;
    border-radius: 10px;
    width: 500px;
    margin: auto;
  }

  input[type="file"] {
    margin: 15px;
  }

  button {
    padding: 10px 20px;
    background: #3498db;
    color: white;
    cursor: pointer;
  }

  button:hover {
    background: #2980b9;
  }

  .result {
    margin-top: 20px;
    font-weight: bold;
  }

  .detected {
    color: red;
  }
```

```

.safe {
  color: green;
}

img {
  margin-top: 20px;
  width: 100%;
  border-radius: 10px;
}
</style>
</head>
<body>

<h1> YOLO vs YOLO + Temporal Consistency</h1>

<div class="card">

  <form method="POST" enctype="multipart/form-data">
    <input type="file" name="video" required>
    <br>
    <button type="submit">Upload & Detect</button>
  </form>

  {% if base_result %}
  <div class="result">
    <p>
      ♦ Base YOLO Result:
      <span class="{{ 'detected' if base_result == 'Detected' else 'safe' }}">
        {{ base_result }}
      </span>
    </p>

    <p>
      ♦ Temporal Consistency Result:
      <span class="{{ 'detected' if temporal_result == 'Confirmed' else 'safe' }}">
        {{ temporal_result }}
      </span>
    </p>
  </div>
  {% endif %}

  {% if image %}
  
  {% endif %}

```

```

</div>

</body>
</html>

```

App.py

```

import os
import cv2

from flask import Flask, render_template, request, redirect, url_for, flash
from ultralytics import YOLO as YOLOModel

# =====
# Initialize Flask Application
# =====
UPLOAD_FOLDER = "static/uploads"
DETECTED_FOLDER = "static/detected"
# =====
# Load YOLO Model
# =====
model = YOLO("best.pt") # Keep your trained model here

CONF_THRESHOLD = 0.5
CONSECUTIVE_FRAMES = 5

# =====
# Home Route
# =====
@app.route("/", methods=["GET", "POST"])
def index():

    if request.method == "POST":

        video = request.files["video"]

        if video.filename == "":
            flash("Please upload a video file!")
            return redirect(request.url)

        video_path = os.path.join(UPLOAD_FOLDER, video.filename)
        video.save(video_path)

```

```

# =====
# BASE YOLO DETECTION
# =====
cap = cv2.VideoCapture(video_path)
frame_number = 0
base_detected = False
temporal_detected = False
detection_counter = 0
detected_image_path = None

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    frame_number += 1

    results = model(frame, conf=CONF_THRESHOLD, verbose=False)
    detections = results[0].boxes

    # -----
    # Base YOLO (Single Frame)
    # -----
    if len(detections) > 0 and not base_detected:
        base_detected = True

    # -----
    # Temporal Consistency
    # -----
    if len(detections) > 0:
        detection_counter += 1
    else:
        detection_counter = 0

    if detection_counter >= CONSECUTIVE_FRAMES:
        temporal_detected = True

    annotated_frame = results[0].plot()
    detected_image_path = os.path.join(DETECTED_FOLDER, "detected.jpg")
    cv2.imwrite(detected_image_path, annotated_frame)
    break

cap.release()

```



```

base_result = "Detected" if base_detected else "Not Detected"
temporal_result = "Confirmed" if temporal_detected else "Rejected"

return render_template(
    "index2.html",
    base_result=base_result,
    temporal_result=temporal_result,
    image="detected/detected.jpg" if temporal_detected else None
)

return render_template("index2.html")

if __name__ == "__main__":
    app.run(debug=True)

```

5.4 OUTPUT SCREENS AND RESULT ANALYSIS

To execute the developed wildlife detection application on a local system, the user must first open the command prompt and navigate to the project directory where the application files are stored. After accessing the project folder, a virtual environment can be created to manage the required dependencies and libraries. The virtual environment ensures that all packages required for the system run independently without affecting other Python installations. The environment can be created using the following command:

```
python -m venv .tfenv
```

Once the command is executed, a folder named “.tfenv” will be created inside the project directory. This folder contains all the necessary Python libraries and dependencies required for running the application.

The virtual environment can then be activated using the following command:

```
.tfenv\Scripts\activate.bat
```

After activating the environment, the web application can be launched. The developed livestock protection system runs as a local web application that allows users to upload surveillance videos and detect harmful wildlife. The application can be started using the command shown below.

```
app.py
```

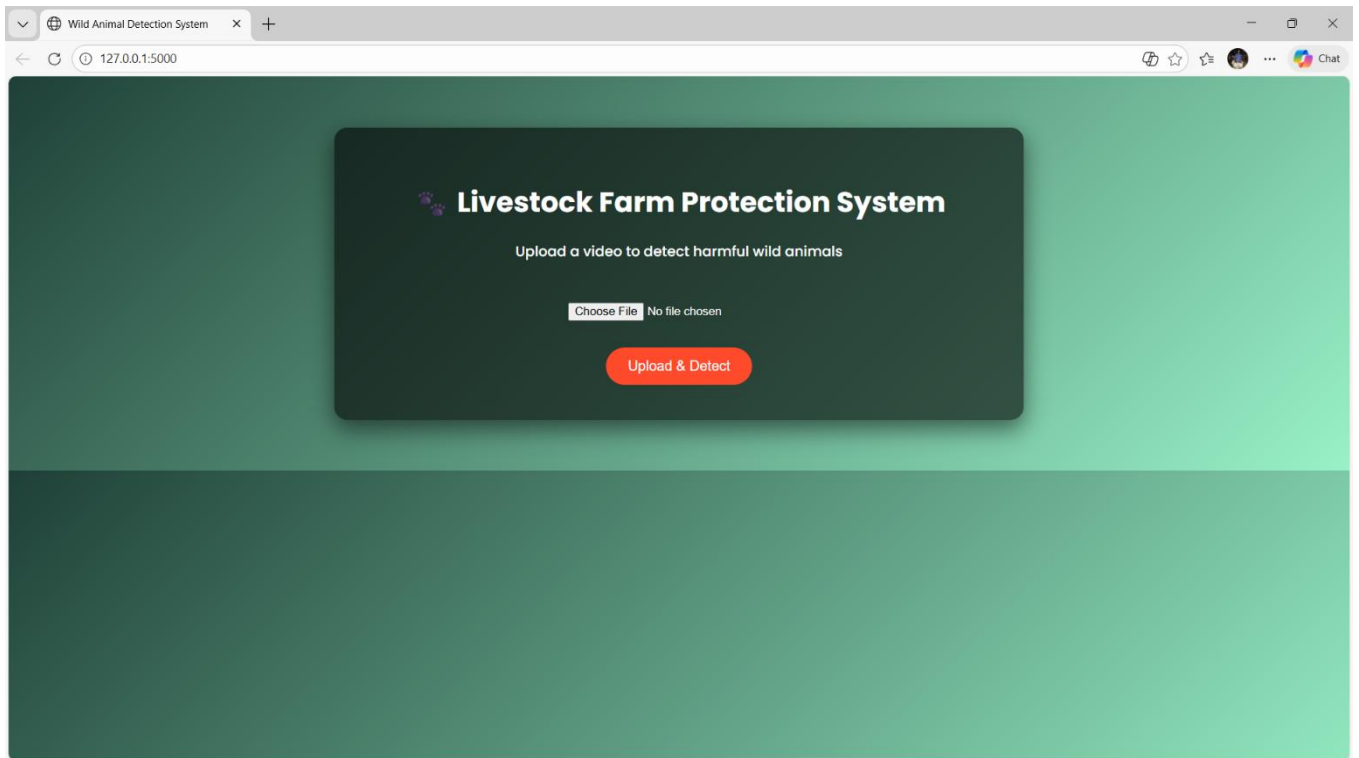


Fig 5.2 User Interface of the Livestock Farm Protection System for Video Upload and Animal Detection

The Fig 5.2 displays the main user interface of the developed livestock farm protection system. The interface provides a simple and interactive environment where users can upload surveillance videos for wildlife detection. The page contains a clear heading indicating the purpose of the application and provides an upload option for selecting video files from the local system. The layout is designed to be user-friendly so that farmers or operators can easily interact with the application without requiring technical expertise. This screen serves as the starting point for performing wildlife detection using the trained YOLOv11 model.

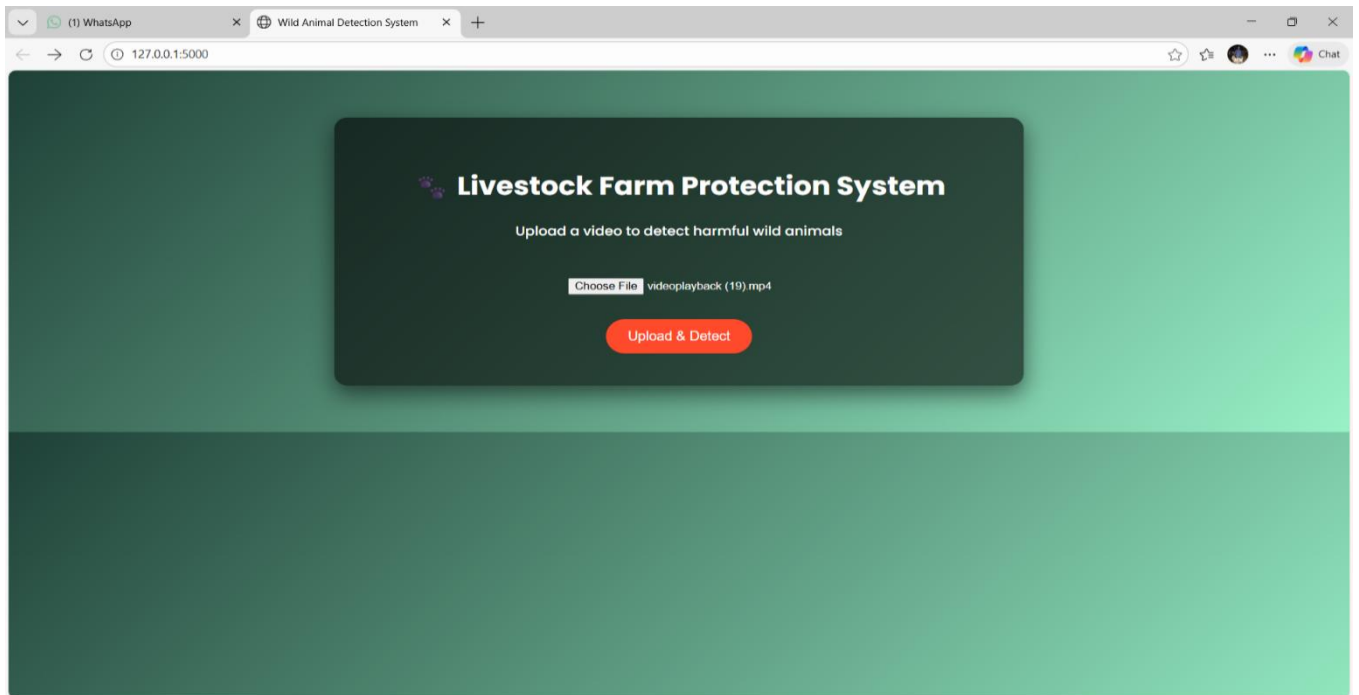


Fig 5.3 Video Upload Interface with Selected Surveillance Video for Wildlife Detection

The Fig 5.3 illustrates the stage where the user selects a surveillance video file for wildlife detection. The chosen file name is displayed on the interface to confirm that the correct video has been selected. After selecting the video file, the user can initiate the detection process by clicking the Upload & Detect button. The system then begins processing the uploaded video by extracting frames and analysing them using the trained YOLOv11 model. This step allows users to verify the selected input before the detection process begins.

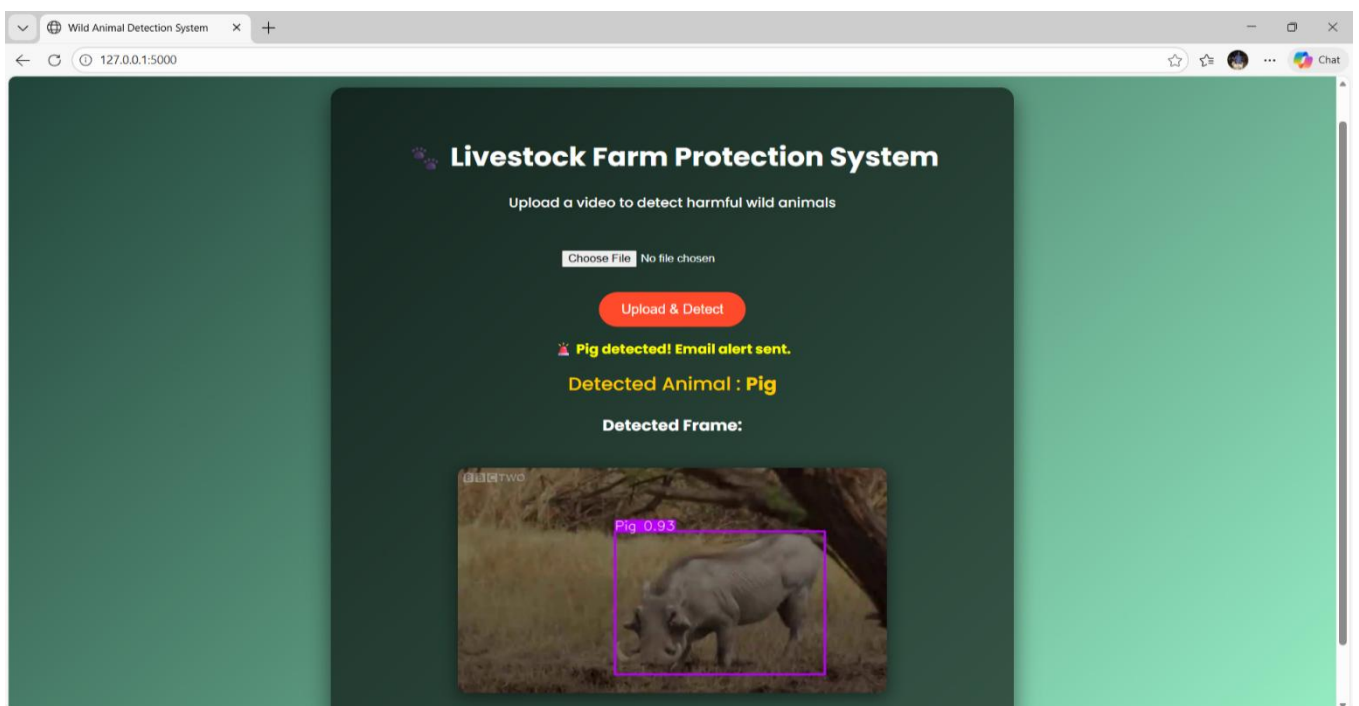


Fig 5.4 Detection Result Showing Identified Wild Animal with Bounding Box and Email Alert Notification

The Fig 5.4 presents the detection result generated by the system after analysing the uploaded surveillance video. A rectangular bounding box is drawn around each identified animal, accompanied by the corresponding class label and prediction confidence score. As seen in the sample output, the model correctly classifies the detected animal as a Pig with an associated confidence value. The system also triggers an automatic email alert to notify the user about the detected wildlife intrusion. This feature enables quick response and improves the safety of livestock farms.

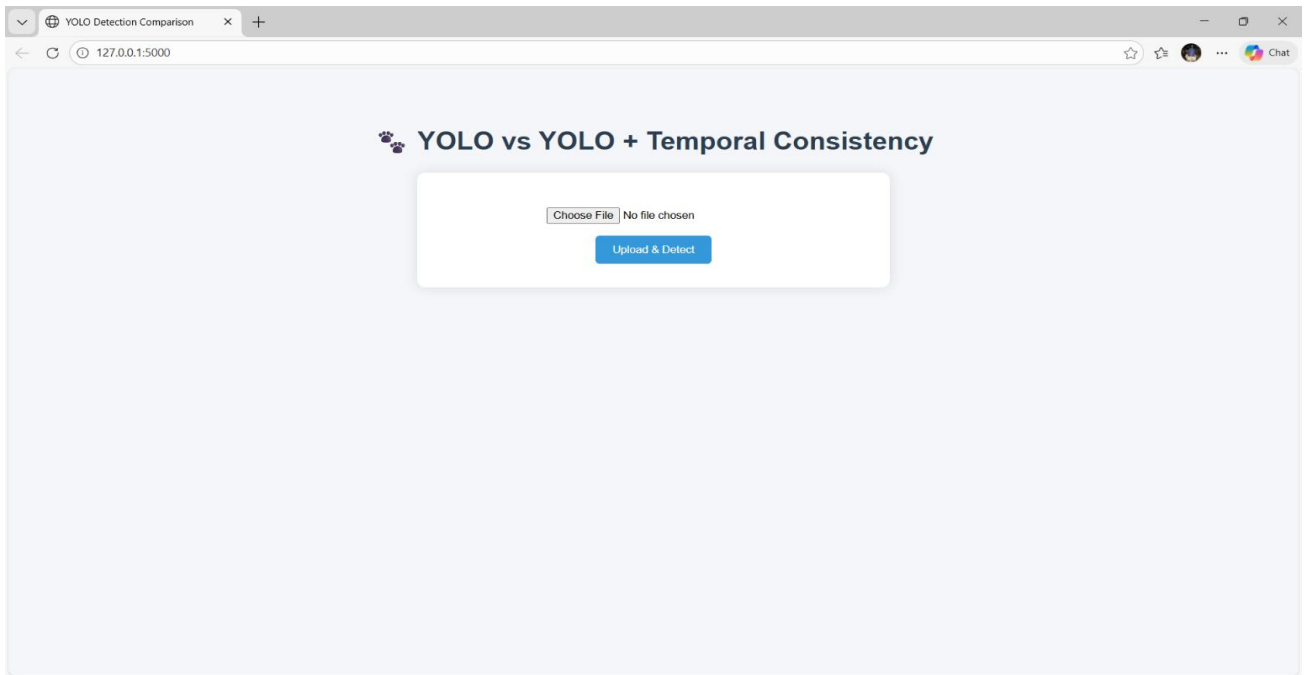


Fig 5.5 YOLO vs YOLO + Temporal Consistency Interface for Video Upload

Figure 5.5 presents the comparison interface created to analyse the differences between the basic YOLO detection model and the enhanced system that incorporates temporal consistency verification. This interface enables users to upload a surveillance video and process it using both detection approaches for comparison. After the video is selected, the system generates detection outputs that allow users to observe variations in detection accuracy, stability, and reliability. The interface layout is similar to the main detection module but is specifically designed to highlight the improvements achieved through temporal verification. This comparison module assists in evaluating the consistency of detections and demonstrates how temporal information improves performance in continuous video monitoring scenarios.

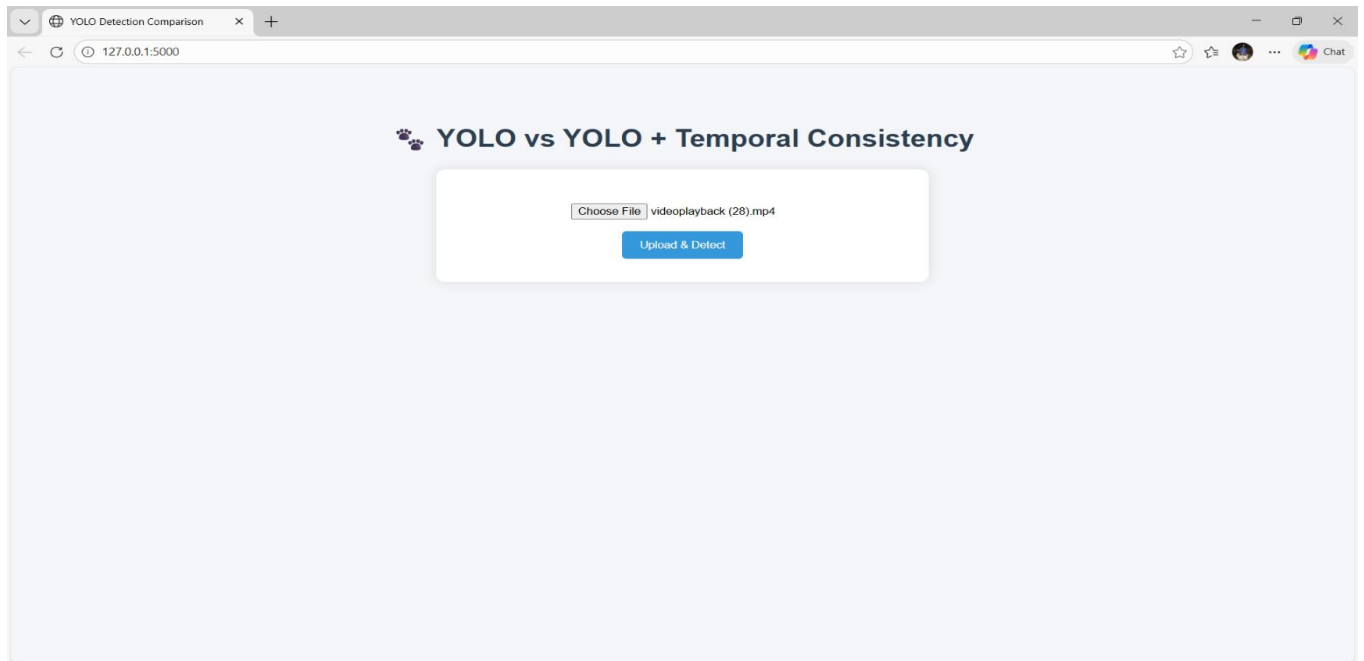


Fig 5.6 Video Selection Interface for YOLO and Temporal Consistency Detection Comparison

Figure 5.6 shows the interface used to select a surveillance video for comparing the performance of the standard YOLO detection model with the enhanced YOLO model that includes temporal consistency. In this screen, the user chooses a video file from the local system using the file selection option. The selected file name is displayed on the interface to confirm the input provided for the comparison process. After selecting the video, the user can initiate the detection process by clicking the Upload & Detect button. This interface helps demonstrate how temporal verification improves the reliability of wildlife detection by validating predictions across multiple video frames.

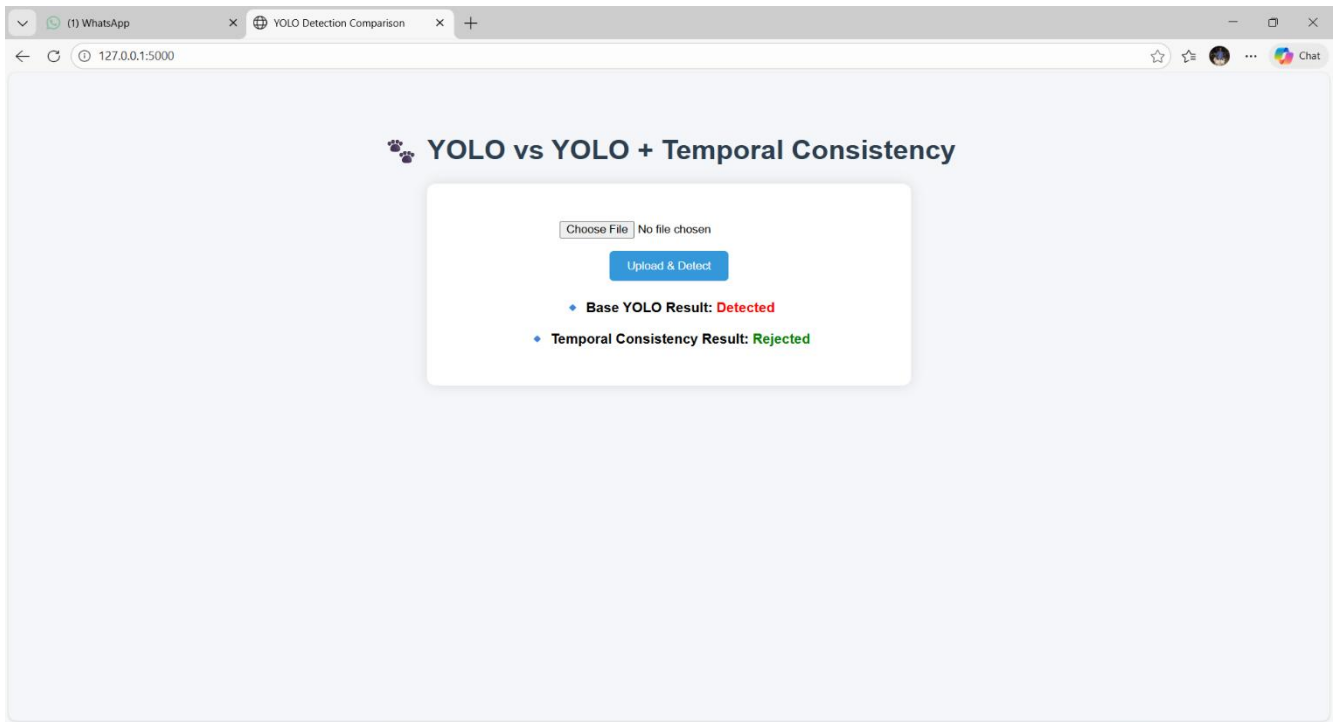


Fig 5.7 Detection Comparison Result Showing Base YOLO Detection and Temporal consistency Verification

The Fig 5.7 illustrates the final comparison results between the standard YOLO detection method and the YOLO model enhanced with temporal consistency. In this example, the base YOLO model detects the presence of an animal in a single frame. However, after applying temporal verification across multiple frames, the detection is rejected because it does not persist consistently over time. This confirms that the temporal consistency mechanism effectively suppresses false detections, thereby improving the dependability of the overall wildlife surveillance system.

5.5 SUMMARY

The implementation stage of the developed wildlife monitoring framework involved building and integrating deep learning–based detection models with video analysis techniques to recognize harmful wild animals in livestock farm environments. Python served as the primary development language, with PyTorch and OpenCV supporting model training and video frame processing respectively. The YOLOv11 architecture was fine-tuned on a custom annotated wildlife dataset to enable accurate multi-class animal identification. During implementation, data preprocessing operations including frame extraction, image resizing, and normalization were applied to the input video streams to make them compatible with the detection model. To enhance detection reliability, a temporal consistency mechanism was also integrated to verify the presence of detected objects across consecutive frames. A

web-based interface was developed to allow farm operators to upload surveillance footage and initiate wildlife detection through an accessible online platform.

Thorough testing and validation were conducted to assess the system's detection capability. The dataset was split into training, validation, and test subsets to evaluate generalization on unseen samples. Detection performance was quantified using precision, recall, F1-score, and mAP as standard evaluation metrics. In addition, the temporal consistency mechanism was examined to confirm its effectiveness in minimizing false detections and reducing detection flickering in continuous video streams. The final system demonstrated strong performance in identifying harmful wild animals and generating timely alerts. The accessible interface combined with stable detection results highlight the practical viability of the proposed framework for deployment in real-world livestock farm surveillance and wildlife intrusion prevention.

CHAPTER-6

TESTING AND VALIDATION

CHAPTER-6

TESTING AND VALIDATION

6.1 INTRODUCTION

The testing and validation phase is essential for evaluating the performance, reliability, and robustness of the proposed Harmful Wild Animal Detection System using YOLOv11 integrated with Temporal Consistency. Following the completion of model training on the augmented dataset, the subsequent phase focused on assessing the model's ability to accurately detect and categorize harmful wild animals across unseen test images and video sequences.

This stage verifies that the trained model operates effectively in practical real-world conditions such as livestock farms and agricultural fields where wildlife intrusion may occur. A structured train-validation-test split was applied to the dataset to ensure unbiased and reliable assessment of the model's detection performance. The trained model is subsequently assessed using previously unseen data to determine how well it can generalize beyond the data used during training.

Detection performance was measured using precision, recall, F1-score, and mAP, collectively assessing the model's ability to correctly identify animals while minimizing false positives and missed detections. In addition, the temporal consistency module is validated to ensure that object detection results remain stable across consecutive frames in video streams. This chapter explains the testing procedures, validation approaches, evaluation metrics, and experimental results used to analyse the performance of the developed wildlife detection system.

6.1.1 INTRODUCTION TO TESTING

Testing is an essential step in the machine learning process, as it verifies whether the trained detection model can accurately perform on previously unseen data. Testing involved feeding unseen images and video frames into the trained YOLOv11 model and comparing the predicted bounding boxes and class labels against their corresponding ground-truth annotations to measure detection accuracy.

The Key objective of the testing phase is to ensure that the system can reliably detect harmful wildlife species while maintaining high detection accuracy and minimizing false positives. The testing process consists of several important steps:

- **Data Preprocessing for Testing:**

To maintain consistency between training and inference, the same preprocessing operations applied during the training phase, specifically image resizing to the required input dimensions

and pixel value normalization, were equally applied to all test samples before feeding them into the model.

- **Model Inference:**

During the testing phase, previously unseen images and video sequences were fed into the trained YOLOv11 model, which generated detection outputs comprising bounding box coordinates, predicted animal class labels, and associated confidence scores for each identified object.

- **Temporal Consistency Validation:**

The predictions generated for consecutive frames are analyzed using the temporal consistency mechanism to verify detection stability and reduce flickering detections.

- **Performance Evaluation:**

To quantitatively assess the detection system, core performance indicators including precision, recall, F1-score, and mean average precision were calculated across all test samples and animal categories.

- **Error Analysis:**

Instances where the model either misidentifies animals or fails to recognize them in difficult situations such as partial occlusion or poor lighting conditions are examined to determine areas that require improvement.

Through systematic testing and validation, the proposed system ensures reliable performance for real-world wildlife monitoring and livestock protection applications.

6.1.2 TRAINING–TESTING DATASET SEPARATION

To properly train and assess the wildlife detection model, the dataset is separated into three subsets following a structured Train–Validation–Test split approach:

- **Training Set (70%) –**

This part of the dataset is utilized for training the YOLOv11 model. During training, the model learns important spatial features of different wild animal classes such as shape, texture, and movement patterns.

- **Validation Set (15%)** –

Throughout training, the validation subset was used to monitor detection performance and optimize hyperparameters including learning rate, batch size, and epoch count, helping prevent overfitting and improving generalization to unseen samples.

- **Test Set (15%)** –

The test subset comprised entirely unseen samples used to obtain an unbiased measure of the final model's detection capability. The dataset split ensured maximum data availability for training while maintaining dedicated subsets for validation and testing. Augmentation techniques including flipping, rotation, brightness adjustment, and noise injection were applied exclusively to the training portion to enhance model robustness without influencing evaluation results.

By implementing this structured 70–15–15 dataset split, the proposed wildlife detection system achieves improved generalization and reliable performance when deployed for real-time monitoring of harmful wild animals in agricultural and livestock environments.

6.2 TEST CASE DESIGN FOR VIDEO

TEST CASE 1: Training only with the image dataset

Table 6.1 Test Case for model training

Case 1	Training the model only with the raw image dataset
Scenario	The YOLOv11 model is trained using the raw image dataset without applying any data augmentation or preprocessing techniques.
Expected Outcome	The model should detect animals in images but may struggle to generalize well to different environments and lighting conditions.
Result	The model showed lower detection accuracy and struggled with complex backgrounds due to the lack of preprocessing and augmentation.

Remark	Test Failed
--------	-------------

TEST CASE 2: Training the model without annotation conversion

Table 6.2 Test Case for Dataset Annotation

Case 2	Train the model without converting annotations to YOLO format
Scenario	The dataset annotations are kept in their original bounding box format instead of converting them into YOLO normalized format.
Expected Outcome	The model should interpret bounding boxes correctly and perform object detection.
Result	The model training process failed because YOLOv11 requires annotation labels to be provided in the normalized YOLO format consisting of class_id, x_center, y_center, width, and height.
Remark	Test Failed

TEST CASE 3: Training the model with proper preprocessing and annotation conversion

Table 6.3 Test Case for Preprocessing and Model Training

Case 3	Training the model with proper preprocessing and annotation conversion
Scenario	Prior to training, all images were resized to the required input dimensions, pixel values were normalized, and bounding box annotations were converted into YOLO-compatible normalized format.
Expected Outcome	Proper preprocessing should improve model learning and enable accurate detection of wild animals.

Result	The preprocessing steps successfully improved the model's learning capability and detection accuracy.
Remark	Test Successful

TEST CASE 4: Training the model without proper data augmentation

Table 6.4 Test Case for Model Generalization

Case 4	Training the model without applying data augmentation techniques
Scenario	Prior to training, all images were resized to the required input dimensions, pixel values were normalized, and bounding box annotations were converted into YOLO-compatible normalized format.
Expected Outcome	While the model demonstrates reliable animal detection, its performance may degrade when deployed in environments significantly different from those represented in the training data.
Result	The model demonstrated limited generalization capability and incorrectly classified animals in difficult scenarios such as changing lighting conditions and partial occlusions.
Remark	Test Failed

TEST CASE 5: Training the model with balanced dataset using augmentation

Table 6.5 Test Case for Dataset Balancing

Case 5	Applying augmentation techniques to balance the dataset
Scenario	Augmentation methods including horizontal flipping, rotation, Gaussian blur, brightness modification, and noise addition are applied to help balance the dataset.
Expected Outcome	Data augmentation should increase dataset diversity and improve model robustness.

Result	The augmented dataset improved detection accuracy and reduced class imbalance issues
Remark	Test Successful.

TEST CASE 6: Testing detection stability without temporal consistency

Table 6.6 Test Case for Video Detection Stability

Case 6	Detection without temporal consistency
Scenario	The YOLOv11 model detects animals frame-by-frame in a video without applying temporal consistency.
Expected Outcome	The system should detect animals but may produce unstable predictions across consecutive frames.
Result	Detection results showed flickering bounding boxes and inconsistent predictions across frames.
Remark	Test Failed

TEST CASE 7: Testing detection with temporal consistency

Table 6.7 Test Case for Temporal Consistency

Case 7	Integrating temporal consistency with YOLOv11
Scenario	The YOLOv11 model predictions are combined with a temporal consistency mechanism that validates detections across multiple frames.
Expected Outcome	Detection results should become stable and reduce flickering across video frames.
Result	Temporal consistency successfully stabilized object detection and improved reliability in video surveillance.
Remark	Test Successful.

TEST CASE 8: Performance of the User Interface

Table 6.8 Test Case for System Interface

Case 8	Real-time wildlife detection through the user interface
Scenario	The user uploads an image or video through the system interface, and the trained YOLOv11 model processes the input and displays detection results.
Expected Outcome	The system is expected to identify animals and display bounding boxes along with predicted class labels and corresponding confidence values.
Result	The application accurately identified animals and presented the results instantaneously, displaying object bounding boxes along with prediction confidence values.
Remark	Test Successful.

6.3 MODEL VALIDATION ON TEST DATASET

Validation testing is essential for evaluating the effectiveness and generalization capability of the harmful wild animal detection system developed using YOLOv11 with temporal consistency. In this study, the dataset is separated into three portions: training (70%), validation (15%), and testing (15%). The validation set is made use of monitor the model's learning progress and fine-tune important training parameters such as the learning rate, batch size, and total epochs. This process ensures that the trained model does not overfit the training dataset and performs well on unseen wildlife images.

During the validation phase, multiple evaluation metrics are used to measure the detection performance of the YOLOv11 model. Precision indicates how accurately the model classifies detected animals, while recall measures the model's ability to identify all relevant animals present in the images. The mAP values provide a detailed evaluation of detection quality by measuring how closely the predicted bounding boxes match the ground-truth annotations.

To improve the model's robustness and minimize overfitting, augmentation techniques such as flipping, rotation, brightness adjustment, and noise addition are applied only to the training dataset. The validation dataset remains unchanged so that it reflects real-world conditions such as different lighting scenarios, background complexity, and animal poses. The YOLOv11 model also uses pretrained weights and optimized training strategies to improve detection accuracy and learning

stability.

By incorporating these validation techniques, the proposed system ensures that the trained model can reliably detect harmful wild animals in diverse environments such as forests, farms, and agricultural fields. The validation results demonstrate strong detection performance across multiple wildlife classes, confirming that the system can effectively identify animals that may pose threats to livestock and crops. The validation outcomes of this project are shown in Table 6.9.

Table 6.9 Prediction outcomes

Class	Precision	Recall	mAP50
0	0.996	1.000	0.995
1	0.846	1.000	0.995
2	1.000	0.988	0.995
3	1.000	0.980	0.995
4	1.000	0.973	0.994
5	0.979	1.000	0.995
6	1.000	0.993	0.995
7	0.997	1.000	0.995
8	1.000	0.953	0.994
9	0.977	0.992	0.994
10	0.998	1.000	0.995
11	1.000	0.957	0.989
12	0.996	0.963	0.975

6.4 PERFORMANCE METRICS ANALYSIS

The YOLOv11 model's performance was assessed through training and validation metrics comprising box loss, classification loss, Distribution Focal Loss, precision, recall, and mAP. All three training losses showed a consistent downward trend across epochs, indicating progressive learning of spatial features and animal-specific patterns. Validation losses followed a similar trajectory, confirming that the model generalized well to unseen data with minimal overfitting.

Alongside loss reduction, precision, recall, and mAP scores improved progressively across training epochs. Precision and recall converged to values between 0.97 and 0.99, reflecting accurate detection with minimal false alarms. The mAP@50 reached approximately 0.99 while mAP@50–95

approached 0.95, demonstrating robust performance across varying IoU thresholds. These results collectively confirm stable training convergence and high detection accuracy, validating the suitability of the proposed system for real-time wildlife surveillance, as illustrated in Figure 6.1.

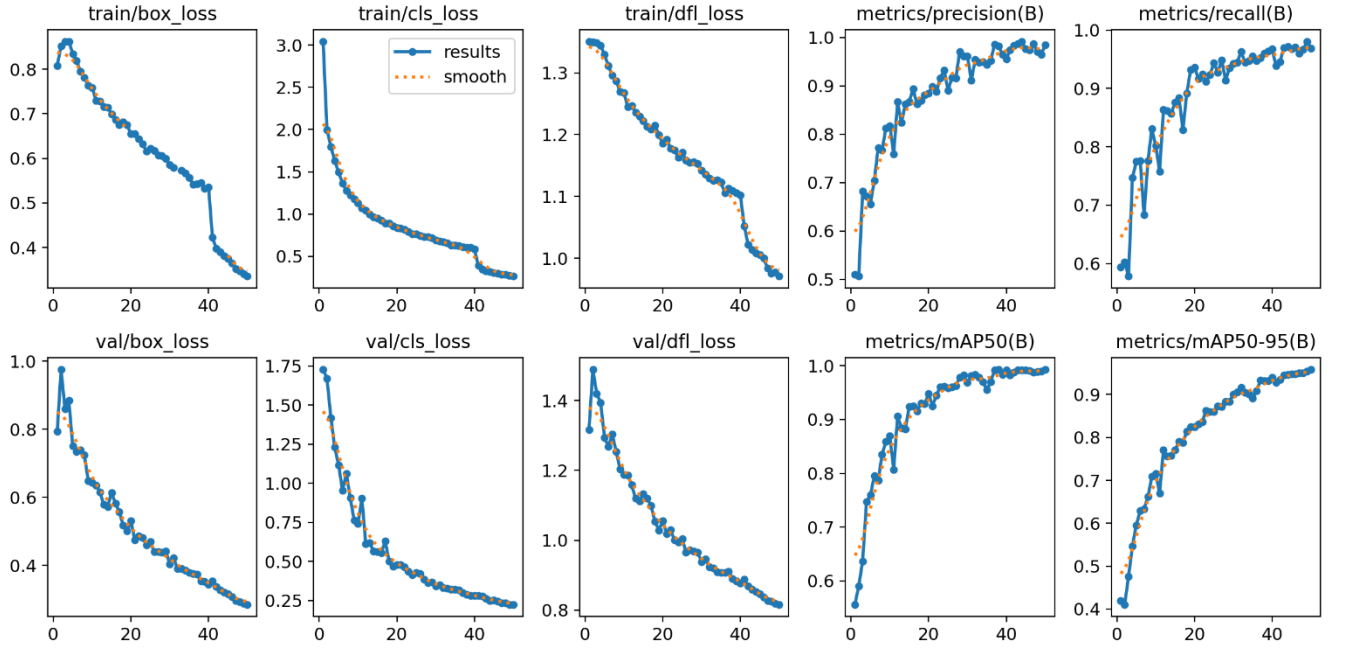


Fig 6.1 YOLOv11 Training and Validation Performance Analysis

6.5 PREDICTION OUTCOMES

6.5.1 YOLOv11 MODEL ANALYSIS

The YOLOv11 model demonstrates strong predictive performance in detecting and classifying wild animals from surveillance video frames. The model successfully detected various species including tigers, elephants, and other wildlife with high confidence, producing accurate bounding boxes and class labels. The YOLOv11 architecture demonstrated the ability to capture animal-specific spatial features across diverse environmental conditions, maintaining reliable detection in complex backgrounds such as forested and agricultural settings. The resulting precision and recall scores confirm the model's effectiveness in correctly identifying animals while keeping both false positives and false negatives to a minimum. However, certain cases where animals are partially occluded or appear at long distances, the model may produce slightly lower confidence scores.

6.5.2 TEMPORAL CONSISTENCY MODULE ANALYSIS

The Temporal Consistency module enhances the reliability of the YOLOv11 detections by analyzing object predictions across consecutive video frames. Instead of relying on a single-frame detection, this module performs object tracking, smoothing, and confirmation to ensure that detected animals remain consistent over time. This significantly reduces false detections caused by environmental noise, shadows, or moving objects such as leaves or livestock. By validating detections across multiple frames, the system confirms the presence of wild animals more accurately. This approach improves overall system stability and ensures that alerts are generated only when a genuine animal intrusion is detected.

6.5.3 EMAIL ALERT SYSTEM ANALYSIS

The Email Alert System acts as the final component in the detection pipeline by notifying users when a harmful wild animal is confirmed. Once the temporal consistency module verifies the detection, the system triggers an alert containing the detected frame, bounding box image, class name, and confidence score. This allows farmers or livestock owners to receive immediate notifications and take preventive actions. The alert mechanism ensures real-time communication and enhances the practicality of the system for field deployment. In cases where detections are not confirmed across frames, the system discards the result to prevent unnecessary alerts.

6.5.4 SYSTEM PERFORMANCE: STRENGTHS AND WEAKNESSES

The developed system combines the YOLOv11 object detection model with a temporal consistency mechanism and an automated email alert feature to deliver an effective approach for wildlife intrusion monitoring in livestock farms. YOLOv11 delivers strong detection accuracy alongside low inference latency, making it well-suited for live surveillance scenarios where rapid identification of harmful wildlife is critical. The temporal consistency module enhances detection reliability by verifying object presence across consecutive video frames, which helps reduce false positives and detection flickering caused by background movements or environmental noise. In addition, the email alert mechanism sends immediate notifications to users whenever a harmful wild animal is detected, allowing farmers or farm managers to take timely preventive actions. This automated notification system improves the practicality and responsiveness of the monitoring framework. However, certain limitations may arise when animals appear under extremely poor lighting conditions or when the surveillance camera resolution is low, which can affect detection accuracy. Despite these challenges, the proposed system provides a reliable and practical approach for monitoring wildlife intrusion and improving the safety and protection of livestock farms.

Table 6.10 Strengths and Weaknesses of System Components

S.No	Component	Performance	Strengths	Weaknesses
1	YOLOv11 Model	High Accuracy (~95–98%)	Fast detection, accurate bounding boxes, real-time capability	Lower confidence in occluded or distant objects
2	Temporal Consistency	High Reliability	Reduces false positives, stable multi-frame detection	Slight delay due to frame verification
3	Email Alert System	Real-time Notification	Immediate alerts with image and detection details	Depends on internet connectivity .36

6.5.5 RESULT ANALYSIS

The result analysis of the proposed Wild Animal Detection System using YOLOv11 with Temporal Consistency demonstrates the effectiveness of the model in identifying harmful wildlife from surveillance video frames. The YOLOv11 object detection model successfully detects animals such as tigers, elephants, snakes, and other wildlife species by generating accurate object bounding boxes, predicted class labels, and associated confidence values for each identified object. The experimental findings indicate that the model operates consistently under various environmental conditions, including open farmland areas and locations near forest boundaries where wildlife movement is common. The detection outputs clearly highlight the identified animals with their corresponding classification labels and confidence values, enabling users to easily interpret the prediction results. The integration of the temporal consistency mechanism further improves detection stability by verifying detections across consecutive frames, thereby reducing false positives and improving the reliability of wildlife monitoring.

To better illustrate the detection performance, the results are presented using grid-based visualization images where each sub-image displays the detected animal along with its predicted bounding box, class name, and confidence score generated by the YOLOv11 model. These visual examples provide a clear understanding of the model's performance across different frames and real-world conditions. Showing multiple detection results together highlights the model's stability and accuracy in identifying wildlife in various scenarios. The prediction outputs and detection results produced by the developed system are presented in Figure 6.2, Figure 6.3, and Figure 6.4, which collectively highlight the model's capability to detect harmful wildlife effectively in livestock farm monitoring environments.

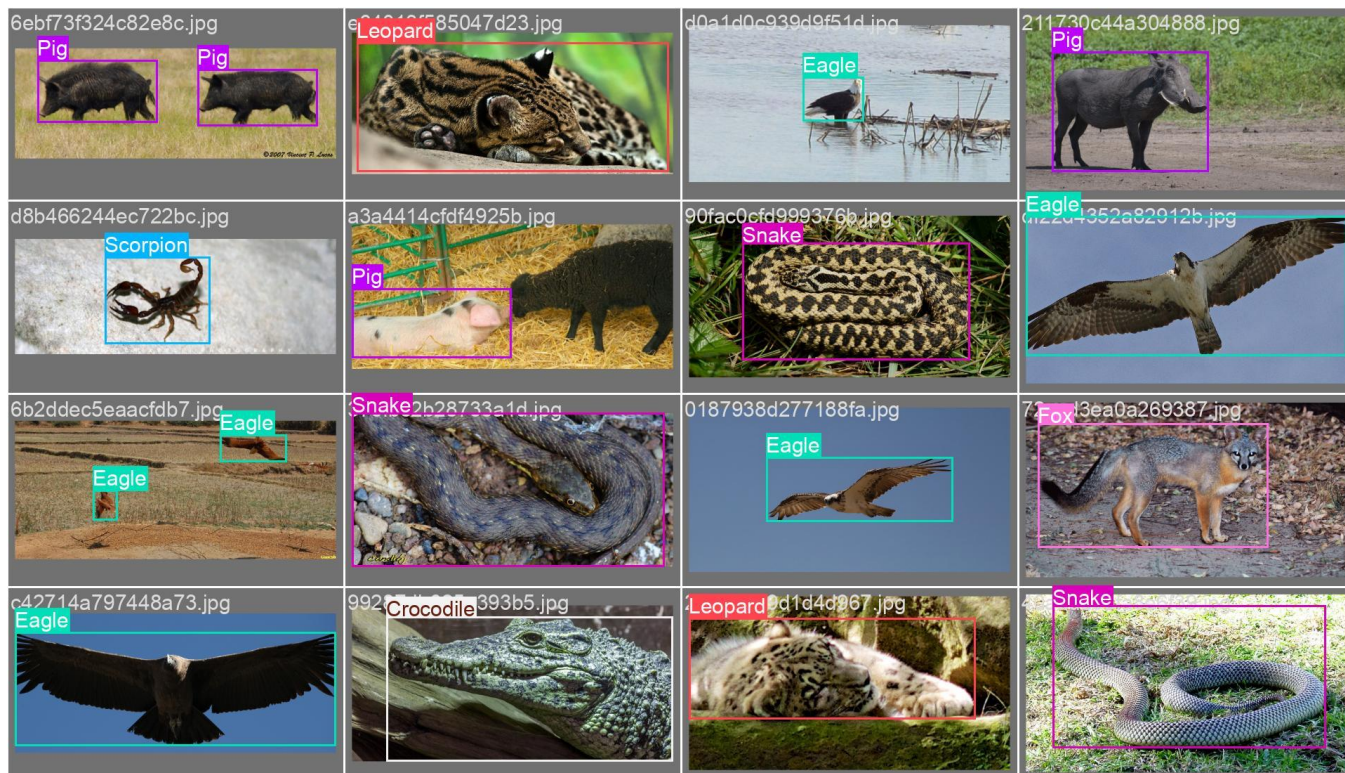


Fig 6.2: Animal Detection Results

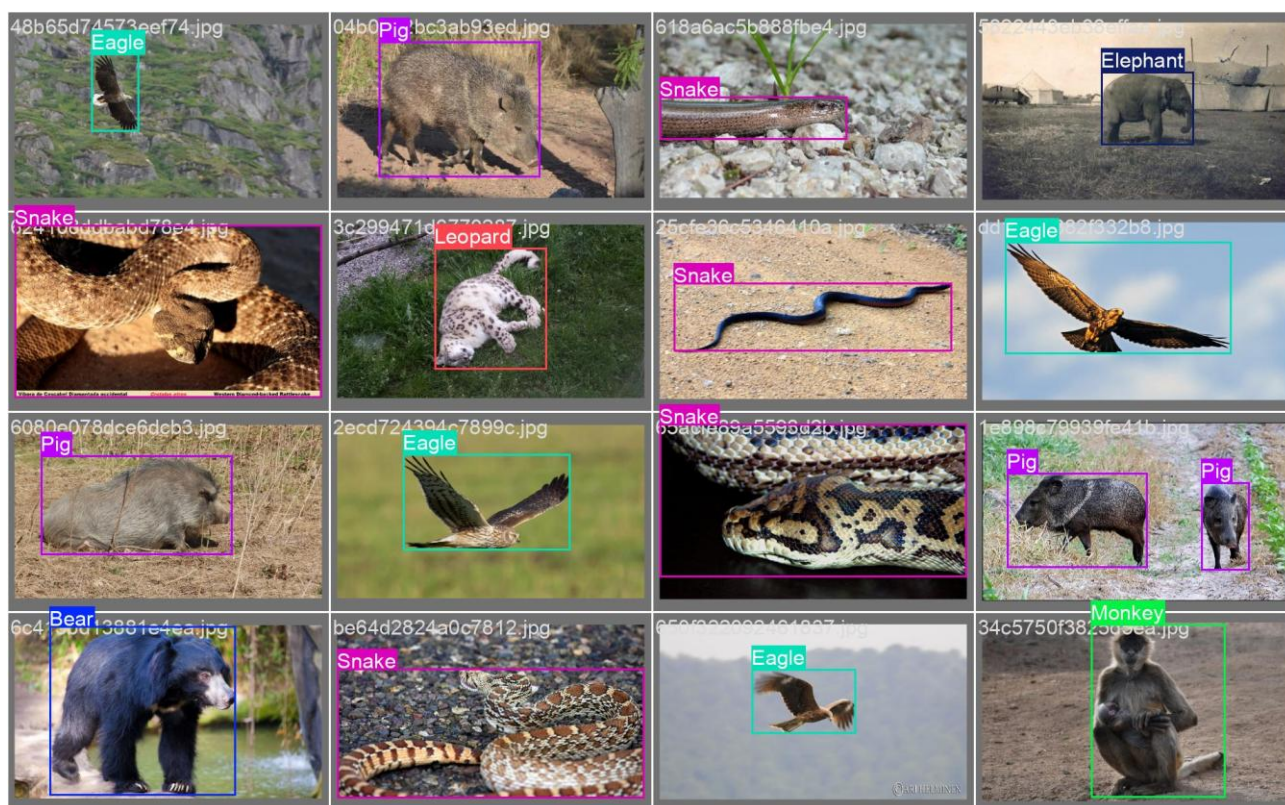


Fig 6.3: Wildlife Ground Truth Labels

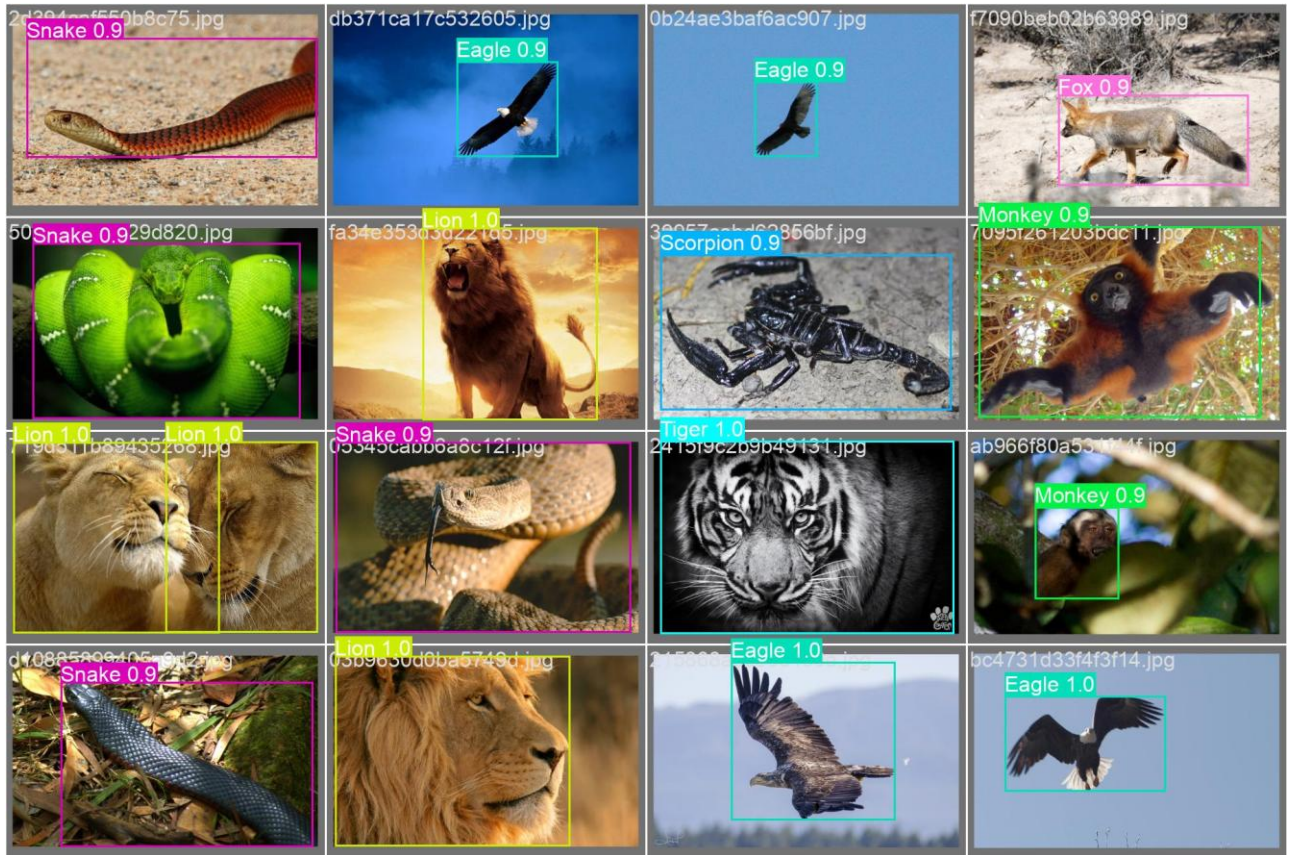


Fig 6.4: Wildlife Ground Truth Labels

6.6 CHAPTER SUMMARY

This project delivers a thorough evaluation of the proposed YOLOv11-based wild animal detection system, focusing on the effectiveness of object detection and temporal consistency in surveillance environments. The system was evaluated through testing on multiple video frames extracted from wildlife monitoring scenarios. The YOLOv11 model was utilized to identify animals and produce bounding boxes, predicted class categories, and associated confidence values. The experimental evaluation monitored detection accuracy, confidence scores, and consistency of predictions across consecutive frames to ensure reliable identification of wild animals.

The detection results indicated that the model could accurately recognize animals under various environmental conditions, including changes in lighting and complex backgrounds. The integration of the temporal consistency module improved detection stability by verifying predictions across multiple frames, which significantly reduced false positives caused by environmental noise or temporary object movements. The bounding box predictions and confidence scores remained consistent across frames, demonstrating the robustness of the system in real-time surveillance applications. However, in certain scenarios such as low-light conditions or partial occlusions, the model occasionally produced lower confidence scores, suggesting that further improvements such as

data augmentation, additional training samples, and parameter tuning could enhance detection reliability.

In conclusion, the proposed YOLOv11-based wild animal detection system with temporal consistency demonstrates strong performance in identifying and monitoring wildlife intrusions. Integrating deep learning detection with temporal verification significantly enhanced system accuracy and dependability, strengthening its suitability for practical deployment. The experimental outcomes confirm that the proposed framework serves as a viable solution for wildlife monitoring and livestock farm security. Prospective improvements may include enhancing detection robustness under challenging lighting and weather conditions, along with expanding the training dataset to broaden species coverage and improve generalization.

CHAPTER-7

CONCLUSION

CHAPTER-7

CONCLUSION

7.1 CONCLUSION

This research presented a real-time surveillance system designed to identify dangerous wild animals, utilizing the YOLOv11 object detection model along with a temporal consistency approach. By utilizing enhanced architectural components, including an optimized backbone and effective multi-scale feature integration, the model delivered strong detection performance across 13 animal categories. Testing on the evaluation dataset yielded strong scores for precision, recall, F1-score, and mean average precision (mAP), indicating that the model achieves high accuracy and reliable detection performance.

Incorporating temporal verification further strengthened the system by minimizing false detections and maintaining consistent predictions across sequential video frames. These characteristics make the framework well-suited for practical deployment in livestock farms and nearby residential zones. Despite its promising performance, challenges such as reduced effectiveness under poor video quality and the limited number of supported classes indicate potential directions for future enhancement. Overall, the proposed solution offers a scalable and efficient deep learning-based approach for intelligent wildlife surveillance, contributing to the reduction of human-wildlife conflicts.

7.2 FUTURE ENHANCEMENTS

To further improve the proposed YOLOv11 with temporal consistency framework, several enhancements can be explored. One important direction is integrating low-light image enhancement techniques or thermal/infrared imaging sensors to improve detection performance in nighttime and extreme dark conditions. Incorporating advanced data collection strategies with more diverse environmental conditions, including fog, rain, and shadow-rich scenes, can further enhance model robustness. Additionally, applying attention mechanisms or transformer-based feature extraction modules may help reduce misclassification among visually similar animal species.

Another promising enhancement involves extending the system from detection to tracking by integrating multi-object tracking algorithms such as Deep SORT to monitor animal movement patterns across frames. Edge optimization techniques, including model pruning and quantization, can further improve deployment efficiency on resource-constrained devices. Finally, integrating an automated alert system with IoT-based sirens, mobile notifications, or GSM modules can transform the framework into a fully autonomous real-time wildlife intrusion prevention system suitable for large-scale farm deployment.

REFERENCES

- [1] S. Delwar *et al.*, “Real-Time Farm Surveillance Using IoT and YOLOv8 for Animal Intrusion Detection,” *Future Internet*, vol. 17, no. 2, p. 70, 2025.
- [2] C. Wang *et al.*, “CF-YOLO for Small Target Detection in Drone Imagery Based on YOLOv11 Algorithm,” *Scientific Reports*, vol. 15, p. 12345, 2025.
- [3] L. Saad Saoud *et al.*, “Real-Time Threaded Houbara Detection and Segmentation for Wildlife Conservation Using YOLOv10 and MobileSAM,” *arXiv preprint arXiv:2510.03501*, 2025.
- [4] J. Kline *et al.*, “Edge-Native Behavior-Adaptive Drone System for Wildlife Monitoring,” *arXiv preprint arXiv:2512.02285*, 2025.
- [5] R. Sapkota and M. Karkee, “Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, and YOLOv8,” *arXiv preprint arXiv:2510.09653*, 2025.
- [6] S. Kumari *et al.*, “Detection of Wild Animal Activity Using Deep Learning Techniques: A Review,” *International Journal of Research in Academic World*, vol. 4, no. 3, pp. 210–222, 2025.
- [7] “Wild Animal Detection and Alarm System,” *Journal of Emerging Technologies and Innovative Research*, vol. 12, no. 4, pp. 102–108, 2025.
- [8] V. K. Jaiswal and H. R. Busi Reddy, “Automated Detection of Large Animals in Road Scene Environments Using Deep Learning,” *International Journal of Innovative Engineering and Emerging Technology*, vol. 13, no. 2, pp. 85–96, 2025.
- [9] “MASF-YOLO: An Improved YOLOv11 Network for Small Object Detection on UAV Imagery,” *arXiv preprint arXiv:2504.18136*, 2025.
- [10] M. Moghadami *et al.*, “Enhancing Small Object Detection with YOLO: A Novel Framework for Improved Accuracy and Efficiency,” *arXiv preprint arXiv:2512.07379*, 2025.
- [11] H. Jung, “YOLO-Drone: Efficient Object Detection Using the Ghost Head Network for Drone Images,” *arXiv preprint arXiv:2511.10905*, 2025.
- [12] H. Peng *et al.*, “YOLOv11-Litchi: Efficient Litchi Fruit Detection Based on UAV Imagery,” *arXiv preprint arXiv:2510.10141*, 2025.
- [13] “Internet of Things-Based Health Surveillance Systems for Livestock,” *IET Smart Systems*, 2025, Art. no. 70013.
- [14] “Automated Camera Trap Depth Estimation & Temporal Data for Wildlife Monitoring,” *ICDICI 2025 Conference Proceedings*, pp. 81–87, 2025.
- [15] S. Kundu *et al.*, “Real-Time Deforestation Anomaly Detection Using YOLO and Lang Chain Agents,” *Scientific Reports*, vol. 15, p. 23617, 2025.
- [16] “Deep Learning for Dynamic Wildlife Monitoring: A Real-Time Approach,” 2025.
- [17] Y. Zhang *et al.*, “ST-YOLO: Spatio-Temporal Enhanced YOLO Framework for Robust Wildlife Detection in Video Streams,” *IEEE Access*, vol. 13, pp. 145632–145645, 2025.
- [18] M. T. Rahman *et al.*, “Real-Time Animal Intrusion Detection Using YOLOv11 with Temporal Smoothing and Edge Deployment,” *Sensors*, vol. 25, no. 4, p. 2145, 2025.
- [19] X. Liu *et al.*, “TCA-YOLO: Temporal Consistency Aware YOLO for Stable Object Detection in Surveillance Videos,” *Expert Systems with Applications*, vol. 258, p. 125321, 2025.
- [20] L. Chen *et al.*, “Video-Based Wildlife Detection Using Hybrid YOLO and Attention Mechanisms for Real-Time Farm Protection,” *Applied Artificial Intelligence*, vol. 39, no. 6, p. 100234, 2025.